

```
1 //油菜种植地块信息模块的 golang 代码
2 package controllers
3 import (
4     "log"
5     "os"
6     "github.com/revel/revel"
7     "github.com/360EntSecGroup-Skylar/excelize/v2"
8     "github.com/jinzhu/gorm"
9     "github.com/jinzhu/gorm/dialects/mysql"
10    "github.com/revel/revel"
11    "github.com/jinzhu/gorm"
12    "github.com/jung-kurt/gofpdf"
13    "your-app-name/app/controllers"
14    "github.com/revel/revel"
15    _ "github.com/jinzhu/gorm/dialects/postgres"
16    "github.com/jinzhu/gorm"
17 )
18 func InitDB() {
19     var err error
20     // Read database configuration from Revel configuration
21     dbConfig := revel.Config.StringDefault("db.url",
22 "postgres://localhost:5432/mydatabase")
23     // Open a connection to the database
24     DB, err = gorm.Open("postgres", dbConfig)
25     if err != nil {
26         revel.AppLog.Fatalf("Failed to connect to database: %v", err)
27     }
28     // Enable logging if needed
29     DB.LogMode(true)
30 }
31 func CloseDB() {
32     if DB != nil {
33         if err := DB.Close(); err != nil {
34             revel.AppLog.Errorf("Failed to close database connection: %v", err)
35         }
36     }
37 }
38 // 油菜种植地块信息管理控制器
39 type crop_fieldController struct {
40     *revel.Controller
41     DB *gorm.DB // 数据库连接对象
42 }
43 // 初始化数据库连接
44 func (c crop_fieldController) Begin() revel.Result {
45     db, err := gorm.Open("mysql",
46 "user:password@dbname?charset=utf8&parseTime=True&loc=Local")
47     if err != nil {
48         revel.AppLog.Error("Failed to connect database")
49         panic(err)
50     }
51     c.DB = db
```

```

52     return nil
53 }
54 // 结束时关闭数据库连接
55 func (c crop_fieldController) End() revel.Result {
56     err := c.DB.Close()
57     if err != nil {
58         revel.AppLog.Error("Failed to close database connection")
59     }
60     return nil
61 }
62 // 列表展示
63 func (c crop_fieldController) List(page int) revel.Result {
64     var crop_field []crop_field
65     c.DB.Offset((page - 1) * PageSize).Limit(PageSize).Find(&crop_field)
66     return c.Render(crop_field)
67 }
68 // 数据添加
69 func (c crop_fieldController) Add(menu crop_field) revel.Result {
70     c.Validation.Required(menu.Field1).Message("地块编号不能为空")
71     c.Validation.Required(menu.Field2).Message("地块名称不能为空")
72     if c.Validation.HasErrors() {
73         c.Validation.Keep()
74         c.FlashParams()
75         return c.Redirect(crop_fieldController.List)
76     }
77     c.DB.Create(&menu)
78     return c.Redirect(crop_fieldController.List)
79 }
80 // 数据编辑
81 func (c crop_fieldController) Edit(menu crop_field) revel.Result {
82     c.DB.Save(&menu)
83     return c.Redirect(crop_fieldController.List)
84 }
85 // 数据删除
86 func (c crop_fieldController) Delete(id int) revel.Result {
87     var menu crop_field
88     c.DB.First(&menu, id)
89     c.DB.Delete(&menu)
90     return c.Redirect(crop_fieldController.List)
91 }
92 // 数据分页
93 const PageSize = 10
94 // 数据查询
95 func (c crop_fieldController) Search(keyword string, page int) revel.Result {
96     var crop_field []crop_field
97     c.DB.Where("字段 1 LIKE ?", "%"+keyword+"%").Offset((page - 1) *
98     PageSize).Limit(PageSize).Find(&crop_field)
99     return c.Render(crop_field)
100 }
101 // Excel 导入
102 func (c crop_fieldController) ImportExcel(file revel.File) revel.Result {

```

```
103     f, err := excelize.OpenReader(file)
104     if err != nil {
105         revel.AppLog.Error("Failed to open Excel file")
106         return c.Redirect(crop_fieldController.List)
107     }
108     // 读取并处理 Excel 数据
109     // TODO: 实现 Excel 数据处理逻辑
110     if file == nil {
111         c.Flash.Error("请选择要导入的 Excel 文件")
112         return c.Redirect(MenuController.List)
113     }
114     // 打开 Excel 文件
115     f, err := excelize.OpenReader(file)
116     if err != nil {
117         revel.AppLog.Errorf("打开 Excel 文件失败: %v", err)
118         c.Flash.Error("打开 Excel 文件失败, 请检查文件格式")
119         return c.Redirect(MenuController.List)
120     }
121     // 获取工作表名称
122     sheetName := f.GetSheetName(1) // 假设只有一个工作表, 索引从 1 开始
123     rows, err := f.GetRows(sheetName)
124     if err != nil {
125         revel.AppLog.Errorf("获取 Excel 工作表行数据失败: %v", err)
126         c.Flash.Error("获取 Excel 工作表行数据失败, 请检查文件内容")
127         return c.Redirect(MenuController.List)
128     }
129     var crop_field []Menu1
130     for rowIndex, row := range rows {
131         if rowIndex == 0 {
132             continue // 跳过表头
133         }
134         if len(row) < 5 {
135             revel.AppLog.Errorf("Excel 行数据格式不正确: %v", row)
136             continue // 跳过格式不正确的行
137         }
138         // 解析 Excel 数据
139         menu := Menu1{
140             序号:    rowIndex, // 假设序号是自动生成的
141             field_id: row[0], // 地块编号
142             field_name: row[1], // 地块名称
143             field_area: row[2], // 地块面积(亩)
144             planting_date: row[3], // 种植日期
145             操作者: "admin", // 假设当前操作者是管理员
146         }
147         // 将数据添加到列表
148         crop_field = append(crop_field, menu)
149     }
150     // 将数据批量插入数据库
151     if err := c.DB.Create(&crop_field).Error; err != nil {
152         revel.AppLog.Errorf("批量插入 Excel 数据到数据库失败: %v", err)
153         c.Flash.Error("导入 Excel 数据到数据库失败, 请稍后重试")
154     }
```

```

154         return c.Redirect(MenuController.List)
155     }
156     c.Flash.Success("成功导入 Excel 数据")
157     return c.Redirect(MenuController.List)
158     return c.Redirect(crop_fieldController.List)
159 }
160 //Excel 数据导出
161 func Excelout() {
162     // 创建一个新的 Excel 文件
163     file := excelize.NewFile()
164     // 定义数据
165     employees := []Employee{
166         {1, "Alice", "Smith", 28, "Manager"},
167         {2, "Bob", "Johnson", 35, "Engineer"},
168         {3, "Charlie", "Wilson", 22, "Intern"},
169     }
170     // 创建第一个工作表
171     sheet1 := "Employees"
172     index1 := file.NewSheet(sheet1)
173     // 设置工作表的列标题
174     titles := map[string]string{
175         "A1": "ID",
176         "B1": "field_id",
177         "C1": "field_name",
178         "D1": "field_area",
179         "E1": "planting_date",
180     }
181     for cell, title := range titles {
182         file.SetCellValue(sheet1, cell, title)
183     }
184     // 写入数据到工作表
185     for i, emp := range employees {
186         rowIndex := i + 2 // 数据从第二行开始
187         file.SetCellValue(sheet1,
188             "A"+excelize.ToAlphaString(1)+excelize.ToSliceIndex(rowIndex), emp.ID)
189         file.SetCellValue(sheet1,
190             "B"+excelize.ToAlphaString(2)+excelize.ToSliceIndex(rowIndex), emp.field_id)
191         file.SetCellValue(sheet1,
192             "C"+excelize.ToAlphaString(3)+excelize.ToSliceIndex(rowIndex), emp.field_name)
193         file.SetCellValue(sheet1,
194             "D"+excelize.ToAlphaString(4)+excelize.ToSliceIndex(rowIndex), emp.field_area)
195         file.SetCellValue(sheet1,
196             "E"+excelize.ToAlphaString(5)+excelize.ToSliceIndex(rowIndex), emp.planting_date)
197         file.SetCellValue(sheet1,
198             "E"+excelize.ToAlphaString(6)+excelize.ToSliceIndex(rowIndex), emp.admin)
199     }
200     // 自动调整列宽
201     for col := 'A'; col <= 'E'; col++ {
202         colStr := string(col)
203         file.SetColWidth(sheet1, colStr, colStr, 15)
204     }

```

```

205 // 设置第一个工作表为活动工作表
206 file.SetActiveSheet(index1)
207 // 创建第二个工作表
208 sheet2 := "MoreData"
209 index2 := file.NewSheet(sheet2)
210
211 // 保存文件
212 filePath := "output.xlsx"
213 if err := file.SaveAs(filePath); err != nil {
214     log.Fatalf("Save Excel file error: %v", err)
215 }
216 // 提示文件已保存
217 log.Printf("Excel file '%s' has been saved successfully.\n", filePath)
218 }
219 // FDF 导出
220 func (c crop_fieldController) ExportFDF() revel.Result {
221     var crop_field []crop_field
222     c.DB.Find(&crop_field)
223     // TODO: 实现 FDF 导出逻辑
224     var crop_field []Menu1
225     if err := c.DB.Find(&crop_field).Error; err != nil {
226         revel.AppLog.Errorf("查询数据库失败: %v", err)
227         c.Flash.Error("查询数据库失败, 请稍后重试")
228         return c.Redirect(Menu1Controller.List)
229     }
230     // 创建 PDF 文档对象
231     pdf := gofpdf.New("P", "mm", "A4", "")
232     pdf.AddPage()
233     // 设置字体
234     pdf.SetFont("Arial", "", 12)
235     // 添加表头
236     pdf.CellFormat(10, 7, "序号", 1, 0, "C", false, 0, "")
237     pdf.CellFormat(30, 7, "field_id", 1, 0, "C", false, 0, "")//地块编号
238     pdf.CellFormat(30, 7, "field_name", 1, 0, "C", false, 0, "")//地块名称
239     pdf.CellFormat(30, 7, "field_area", 1, 0, "C", false, 0, "")//地块面积(亩)
240     pdf.CellFormat(30, 7, "planting_date", 1, 1, "C", false, 0, "")//种植日期
241     // 添加数据行
242     for _, menu := range crop_field {
243         pdf.CellFormat(10, 7, strconv.Itoa(int(menu.序号)), 1, 0, "C", false, 0, "")
244         pdf.CellFormat(30, 7, menu.field_id, 1, 0, "", false, 0, "")//地块编号
245         pdf.CellFormat(30, 7, menu.field_name, 1, 0, "", false, 0, "")//地块名称
246         pdf.CellFormat(30, 7, menu.field_area, 1, 0, "", false, 0, "")//地块面积(亩)
247         pdf.CellFormat(30, 7, menu.planting_date, 1, 1, "", false, 0, "")//种植日
248     }
249     // 保存 PDF 文件
250     filePath := "path/to/save/yourfile.pdf"
251     err := pdf.OutputFileAndClose(filePath)
252     if err != nil {
253         revel.AppLog.Errorf("保存 PDF 文件失败: %v", err)
254         c.Flash.Error("保存 PDF 文件失败, 请稍后重试")
255         return c.Redirect(Menu1Controller.List)

```

```

256     }
257     // 返回 PDF 文件路径供下载
258     c.Response.ContentType = "application/pdf"
259     c.Response.Out.Header().Set("Content-Disposition", "attachment;
260 filename="+filePath)
261     return c.File(filePath)
262     return c.Render(crop_field)
263 }
264 package models
265 import "github.com/jinzhu/gorm"
266 // 定义菜单 1 数据模型
267 type crop_field struct {
268     gorm.Model
269     序号      int      `gorm:"column:序号;not null"`
270     field_id   string `gorm:"column:field_id;not null"`//地块编号
271     field_name string `gorm:"column:field_name;not null"`//地块名称
272     field_area string `gorm:"column:field_area"`//地块面积(亩)
273     planting_date string `gorm:"column:字段拼音 4"`//种植日期
274     操作者     string `gorm:"column:操作者"`
275 }
276 func init() {
277     // 路由映射
278     revel.Router.
279         Route("/crop_field/list/{page}", &controllers.crop_fieldController{}).
280         Route("/crop_field/add", &controllers.crop_fieldController{}).
281         Route("/crop_field/edit", &controllers.crop_fieldController{}).
282         Route("/crop_field/delete/{id}", &controllers.crop_fieldController{}).
283         Route("/crop_field/search/{keyword}/{page}",
284 &controllers.crop_fieldController{}).
285         Route("/crop_field/import", &controllers.crop_fieldController{}).
286         Route("/crop_field/export", &controllers.crop_fieldController{}).
287 }
288 //植保无人机设备信息模块的 golang 代码
289 package controllers
290 import (
291     "log"
292     "os"
293     "github.com/revel/revel"
294     "github.com/360EntSecGroup-Skylar/excelize/v2"
295     "github.com/jinzhu/gorm"
296     "github.com/jinzhu/gorm/dialects/mysql"
297     "github.com/revel/revel"
298     "github.com/jinzhu/gorm"
299     "github.com/jung-kurt/gofpdf"
300     "your-app-name/app/controllers"
301     "github.com/revel/revel"
302     _ "github.com/jinzhu/gorm/dialects/postgres"
303     "github.com/jinzhu/gorm"
304 )
305 func InitDB() {
306     var err error

```

```

307 // Read database configuration from Revel configuration
308 dbConfig := revel.Config.StringDefault("db.url",
309 "postgres://localhost:5432/mydatabase")
310 // Open a connection to the database
311 DB, err = gorm.Open("postgres", dbConfig)
312 if err != nil {
313     revel.AppLog.Fatalf("Failed to connect to database: %v", err)
314 }
315 // Enable logging if needed
316 DB.LogMode(true)
317 }
318 func CloseDB() {
319     if DB != nil {
320         if err := DB.Close(); err != nil {
321             revel.AppLog.Errorf("Failed to close database connection: %v", err)
322         }
323     }
324 }
325 // 植保无人机设备信息管理控制器
326 type drone_infoController struct {
327     *revel.Controller
328     DB *gorm.DB // 数据库连接对象
329 }
330 // 初始化数据库连接
331 func (c drone_infoController) Begin() revel.Result {
332     db, err := gorm.Open("mysql",
333 "user:password@/dbname?charset=utf8&parseTime=True&loc=Local")
334     if err != nil {
335         revel.AppLog.Error("Failed to connect database")
336         panic(err)
337     }
338     c.DB = db
339     return nil
340 }
341 // 结束时关闭数据库连接
342 func (c drone_infoController) End() revel.Result {
343     err := c.DB.Close()
344     if err != nil {
345         revel.AppLog.Error("Failed to close database connection")
346     }
347     return nil
348 }
349 // 列表展示
350 func (c drone_infoController) List(page int) revel.Result {
351     var drone_info []drone_info
352     c.DB.Offset((page - 1) * PageSize).Limit(PageSize).Find(&drone_info)
353     return c.Render(drone_info)
354 }
355 // 数据添加
356 func (c drone_infoController) Add(menu drone_info) revel.Result {
357     c.Validation.Required(menu.Field1).Message("无人机编号不能为空")

```

```

358     c.Validation.Required(menu.Field2).Message("型号名称不能为空")
359     if c.Validation.HasErrors() {
360         c.Validation.Keep()
361         c.FlashParams()
362         return c.Redirect(drone_infoController.List)
363     }
364     c.DB.Create(&menu)
365     return c.Redirect(drone_infoController.List)
366 }
367 // 数据编辑
368 func (c drone_infoController) Edit(menu drone_info) revel.Result {
369     c.DB.Save(&menu)
370     return c.Redirect(drone_infoController.List)
371 }
372 // 数据删除
373 func (c drone_infoController) Delete(id int) revel.Result {
374     var menu drone_info
375     c.DB.First(&menu, id)
376     c.DB.Delete(&menu)
377     return c.Redirect(drone_infoController.List)
378 }
379 // 数据分页
380 const PageSize = 10
381 // 数据查询
382 func (c drone_infoController) Search(keyword string, page int) revel.Result {
383     var drone_info []drone_info
384     c.DB.Where(" 字 段 1 LIKE ?", "%"+keyword+"%").Offset((page - 1) *
385     PageSize).Limit(PageSize).Find(&drone_info)
386     return c.Render(drone_info)
387 }
388 // Excel 导入
389 func (c drone_infoController) ImportExcel(file revel.File) revel.Result {
390     f, err := excelize.OpenReader(file)
391     if err != nil {
392         revel.AppLog.Error("Failed to open Excel file")
393         return c.Redirect(drone_infoController.List)
394     }
395     // 读取并处理 Excel 数据
396     // TODO: 实现 Excel 数据处理逻辑
397     if file == nil {
398         c.Flash.Error("请选择要导入的 Excel 文件")
399         return c.Redirect(MenuController.List)
400     }
401     // 打开 Excel 文件
402     f, err := excelize.OpenReader(file)
403     if err != nil {
404         revel.AppLog.Errorf("打开 Excel 文件失败: %v", err)
405         c.Flash.Error("打开 Excel 文件失败, 请检查文件格式")
406         return c.Redirect(MenuController.List)
407     }
408     // 获取工作表名称

```



```

409     sheetName := f.GetSheetName(1) // 假设只有一个工作表, 索引从 1 开始
410     rows, err := f.GetRows(sheetName)
411     if err != nil {
412         revel.AppLog.Errorf("获取 Excel 工作表行数据失败: %v", err)
413         c.Flash.Error("获取 Excel 工作表行数据失败, 请检查文件内容")
414         return c.Redirect(MenuController.List)
415     }
416     var drone_info []Menu1
417     for rowIndex, row := range rows {
418         if rowIndex == 0 {
419             continue // 跳过表头
420         }
421         if len(row) < 5 {
422             revel.AppLog.Errorf("Excel 行数据格式不正确: %v", row)
423             continue // 跳过格式不正确的行
424         }
425         // 解析 Excel 数据
426         menu := Menu1{
427             序号:    rowIndex, // 假设序号是自动生成的
428             drone_id: row[0], // 无人机编号
429             model_name: row[1], // 型号名称
430             tank_capacity: row[2], // 药箱容量(升)
431             flight_time: row[3], // 续航时间(分钟)
432             操作者: "admin", // 假设当前操作者是管理员
433         }
434         // 将数据添加到列表
435         drone_info = append(drone_info, menu)
436     }
437     // 将数据批量插入数据库
438     if err := c.DB.Create(&drone_info).Error; err != nil {
439         revel.AppLog.Errorf("批量插入 Excel 数据到数据库失败: %v", err)
440         c.Flash.Error("导入 Excel 数据到数据库失败, 请稍后重试")
441         return c.Redirect(MenuController.List)
442     }
443     c.Flash.Success("成功导入 Excel 数据")
444     return c.Redirect(MenuController.List)
445     return c.Redirect(drone_infoController.List)
446 }
447 // Excel 数据导出
448 func Excelout() {
449     // 创建一个新的 Excel 文件
450     file := excelize.NewFile()
451     // 定义数据
452     employees := []Employee{
453         {1, "Alice", "Smith", 28, "Manager"},
454         {2, "Bob", "Johnson", 35, "Engineer"},
455         {3, "Charlie", "Wilson", 22, "Intern"},
456     }
457     // 创建第一个工作表
458     sheet1 := "Employees"
459     index1 := file.NewSheet(sheet1)

```

```

460 // 设置工作表的列标题
461 titles := map[string]string{
462     "A1": "ID",
463     "B1": "drone_id",
464     "C1": "model_name",
465     "D1": "tank_capacity",
466     "E1": "flight_time",
467 }
468 for cell, title := range titles {
469     file.SetCellValue(sheet1, cell, title)
470 }
471 // 写入数据到工作表
472 for i, emp := range employees {
473     rowIndex := i + 2 // 数据从第二行开始
474     file.SetCellValue(sheet1,
475 "A"+excelize.ToAlphaString(1)+excelize.ToSliceIndex(rowIndex), emp.ID)
476     file.SetCellValue(sheet1,
477 "B"+excelize.ToAlphaString(2)+excelize.ToSliceIndex(rowIndex), emp.drone_id)
478     file.SetCellValue(sheet1,
479 "C"+excelize.ToAlphaString(3)+excelize.ToSliceIndex(rowIndex), emp.model_name)
480     file.SetCellValue(sheet1,
481 "D"+excelize.ToAlphaString(4)+excelize.ToSliceIndex(rowIndex), emp.tank_capacity)
482     file.SetCellValue(sheet1,
483 "E"+excelize.ToAlphaString(5)+excelize.ToSliceIndex(rowIndex), emp.flight_time)
484 file.SetCellValue(sheet1,
485 "E"+excelize.ToAlphaString(6)+excelize.ToSliceIndex(rowIndex), emp.admin)
486 }
487 // 自动调整列宽
488 for col := 'A'; col <= 'E'; col++ {
489     colStr := string(col)
490     file.SetColWidth(sheet1, colStr, colStr, 15)
491 }
492 // 设置第一个工作表为活动工作表
493 file.SetActiveSheet(index1)
494 // 创建第二个工作表
495 sheet2 := "MoreData"
496 index2 := file.NewSheet(sheet2)
497
498 // 保存文件
499 filePath := "output.xlsx"
500 if err := file.SaveAs(filePath); err != nil {
501     log.Fatalf("Save Excel file error: %v", err)
502 }
503 // 提示文件已保存
504 log.Printf("Excel file '%s' has been saved successfully.\n", filePath)
505 }
506 // FDF 导出
507 func (c drone_infoController) ExportFDF() revel.Result {
508     var drone_info []drone_info
509     c.DB.Find(&drone_info)
510     // TODO: 实现 FDF 导出逻辑

```

```

511     var drone_info []Menu1
512     if err := c.DB.Find(&drone_info).Error; err != nil {
513         revel.AppLog.Errorf("查询数据库失败: %v", err)
514         c.Flash.Error("查询数据库失败, 请稍后重试")
515         return c.Redirect(Menu1Controller.List)
516     }
517     // 创建 PDF 文档对象
518     pdf := gofpdf.New("P", "mm", "A4", "")
519     pdf.AddPage()
520     // 设置字体
521     pdf.SetFont("Arial", "", 12)
522     // 添加表头
523     pdf.CellFormat(10, 7, "序号", 1, 0, "C", false, 0, "")
524     pdf.CellFormat(30, 7, "drone_id", 1, 0, "C", false, 0, "")//无人机编号
525     pdf.CellFormat(30, 7, "model_name", 1, 0, "C", false, 0, "")//型号名称
526     pdf.CellFormat(30, 7, "tank_capacity", 1, 0, "C", false, 0, "")//药箱容量(升)
527     pdf.CellFormat(30, 7, "flight_time", 1, 1, "C", false, 0, "")//续航时间(分钟)
528     // 添加数据行
529     for _, menu := range drone_info {
530         pdf.CellFormat(10, 7, strconv.Itoa(int(menu.序号)), 1, 0, "C", false, 0, "")
531         pdf.CellFormat(30, 7, menu.drone_id, 1, 0, "", false, 0, "")//无人机编号
532         pdf.CellFormat(30, 7, menu.model_name, 1, 0, "", false, 0, "")//型号名称
533         pdf.CellFormat(30, 7, menu.tank_capacity, 1, 0, "", false, 0, "")//药箱容量(升)
534         pdf.CellFormat(30, 7, menu.flight_time, 1, 1, "", false, 0, "")//续航时间(分
535 钟)    }
536     // 保存 PDF 文件
537     filePath := "path/to/save/yourfile.pdf"
538     err := pdf.OutputFileAndClose(filePath)
539     if err != nil {
540         revel.AppLog.Errorf("保存 PDF 文件失败: %v", err)
541         c.Flash.Error("保存 PDF 文件失败, 请稍后重试")
542         return c.Redirect(Menu1Controller.List)
543     }
544     // 返回 PDF 文件路径供下载
545     c.Response.ContentType = "application/pdf"
546     c.Response.Out.Header().Set("Content-Disposition", "attachment;
547 filename="+filePath)
548     return c.File(filePath)
549     return c.Render(drone_info)
550 }
551 package models
552 import "github.com/jinzhu/gorm"
553 // 定义菜单 1 数据模型
554 type drone_info struct {
555     gorm.Model
556     序号      int      `gorm:"column:序号;not null"`
557     drone_id   string   `gorm:"column:drone_id;not null"`//无人机编号
558     model_name string   `gorm:"column:model_name;not null"`//型号名称
559     tank_capacity string `gorm:"column:tank_capacity"`//药箱容量(升)
560     flight_time string `gorm:"column:字段拼音 4"`//续航时间(分钟)
561     操作者     string   `gorm:"column:操作者"`

```

```

562 }
563 func init() {
564     // 路由映射
565     revel.Router.
566         Route("/drone_info/list/{page}", &controllers.drone_infoController{}).
567         Route("/drone_info/add", &controllers.drone_infoController{}).
568         Route("/drone_info/edit", &controllers.drone_infoController{}).
569         Route("/drone_info/delete/{id}", &controllers.drone_infoController{}).
570         Route("/drone_info/search/{keyword}/{page}",
571 &controllers.drone_infoController{}).
572         Route("/drone_info/import", &controllers.drone_infoController{}).
573         Route("/drone_info/export", &controllers.drone_infoController{}).
574 }
575 //农药信息配置管理模块的 go lang 代码
576 package controllers
577 import (
578     "log"
579     "os"
580     "github.com/revel/revel"
581     "github.com/360EntSecGroup-Skylar/excelize/v2"
582     "github.com/jinzhu/gorm"
583     "github.com/jinzhu/gorm/dialects/mysql"
584     "github.com/revel/revel"
585     "github.com/jinzhu/gorm"
586     "github.com/jung-kurt/gofpdf"
587     "your-app-name/app/controllers"
588     "github.com/revel/revel"
589     _ "github.com/jinzhu/gorm/dialects/postgres"
590     "github.com/jinzhu/gorm"
591 )
592 func InitDB() {
593     var err error
594     // Read database configuration from Revel configuration
595     dbConfig := revel.Config.StringDefault("db.url",
596 "postgres://localhost:5432/mydatabase")
597     // Open a connection to the database
598     DB, err = gorm.Open("postgres", dbConfig)
599     if err != nil {
600         revel.AppLog.Fatalf("Failed to connect to database: %v", err)
601     }
602     // Enable logging if needed
603     DB.LogMode(true)
604 }
605 func CloseDB() {
606     if DB != nil {
607         if err := DB.Close(); err != nil {
608             revel.AppLog.Errorf("Failed to close database connection: %v", err)
609         }
610     }
611 }
612 // 农药信息配置管理管理控制器

```

```

613 type pesticide_infoController struct {
614     *revel.Controller
615     DB *gorm.DB // 数据库连接对象
616 }
617 // 初始化数据库连接
618 func (c pesticide_infoController) Begin() revel.Result {
619     db, err := gorm.Open("mysql",
620     "user:password@/dbname?charset=utf8&parseTime=True&loc=Local")
621     if err != nil {
622         revel.AppLog.Error("Failed to connect database")
623         panic(err)
624     }
625     c.DB = db
626     return nil
627 }
628 // 结束时关闭数据库连接
629 func (c pesticide_infoController) End() revel.Result {
630     err := c.DB.Close()
631     if err != nil {
632         revel.AppLog.Error("Failed to close database connection")
633     }
634     return nil
635 }
636 // 列表展示
637 func (c pesticide_infoController) List(page int) revel.Result {
638     var pesticide_info []pesticide_info
639     c.DB.Offset((page - 1) * PageSize).Limit(PageSize).Find(&pesticide_info)
640     return c.Render(pesticide_info)
641 }
642 // 数据添加
643 func (c pesticide_infoController) Add(menu pesticide_info) revel.Result {
644     c.Validation.Required(menu.Field1).Message("农药编号不能为空")
645     c.Validation.Required(menu.Field2).Message("农药名称不能为空")
646     if c.Validation.HasErrors() {
647         c.Validation.Keep()
648         c.FlashParams()
649         return c.Redirect(pesticide_infoController.List)
650     }
651     c.DB.Create(&menu)
652     return c.Redirect(pesticide_infoController.List)
653 }
654 // 数据编辑
655 func (c pesticide_infoController) Edit(menu pesticide_info) revel.Result {
656     c.DB.Save(&menu)
657     return c.Redirect(pesticide_infoController.List)
658 }
659 // 数据删除
660 func (c pesticide_infoController) Delete(id int) revel.Result {
661     var menu pesticide_info
662     c.DB.First(&menu, id)
663     c.DB.Delete(&menu)

```

```

664     return c.Redirect(pesticide_infoController.List)
665 }
666 // 数据分页
667 const PageSize = 10
668 // 数据查询
669 func (c pesticide_infoController) Search(keyword string, page int) revel.Result {
670     var pesticide_info []pesticide_info
671     c.DB.Where(" 字 段 1 LIKE ?", "%"+keyword+"%").Offset((page - 1) *
672     PageSize).Limit(PageSize).Find(&pesticide_info)
673     return c.Render(pesticide_info)
674 }
675 // Excel 导入
676 func (c pesticide_infoController) ImportExcel(file revel.File) revel.Result {
677     f, err := excelize.OpenReader(file)
678     if err != nil {
679         revel.AppLog.Error("Failed to open Excel file")
680         return c.Redirect(pesticide_infoController.List)
681     }
682     // 读取并处理 Excel 数据
683     // TODO: 实现 Excel 数据处理逻辑
684     if file == nil {
685         c.Flash.Error("请选择要导入的 Excel 文件")
686         return c.Redirect(MenuController.List)
687     }
688     // 打开 Excel 文件
689     f, err := excelize.OpenReader(file)
690     if err != nil {
691         revel.AppLog.Errorf("打开 Excel 文件失败: %v", err)
692         c.Flash.Error("打开 Excel 文件失败, 请检查文件格式")
693         return c.Redirect(MenuController.List)
694     }
695     // 获取工作表名称
696     sheetName := f.GetSheetName(1) // 假设只有一个工作表, 索引从 1 开始
697     rows, err := f.GetRows(sheetName)
698     if err != nil {
699         revel.AppLog.Errorf("获取 Excel 工作表行数据失败: %v", err)
700         c.Flash.Error("获取 Excel 工作表行数据失败, 请检查文件内容")
701         return c.Redirect(MenuController.List)
702     }
703     var pesticide_info []Menu1
704     for rowIndex, row := range rows {
705         if rowIndex == 0 {
706             continue // 跳过表头
707         }
708         if len(row) < 5 {
709             revel.AppLog.Errorf("Excel 行数据格式不正确: %v", row)
710             continue // 跳过格式不正确的行
711         }
712         // 解析 Excel 数据
713         menu := Menu1{
714             序号:    rowIndex, // 假设序号是自动生成的

```

```

715         pesticide_id: row[0], //农药编号
716         pesticide_name: row[1], //农药名称
717         application_rate: row[2], //施用比例(ml/亩)
718         dilution_ratio: row[3], //稀释比例
719         操作者: "admin", // 假设当前操作者是管理员
720     }
721     // 将数据添加到列表
722     pesticide_info = append(pesticide_info, menu)
723 }
724 // 将数据批量插入数据库
725 if err := c.DB.Create(&pesticide_info).Error; err != nil {
726     revel.AppLog.Errorf("批量插入 Excel 数据到数据库失败: %v", err)
727     c.Flash.Error("导入 Excel 数据到数据库失败, 请稍后重试")
728     return c.Redirect(Menu1Controller.List)
729 }
730 c.Flash.Success("成功导入 Excel 数据")
731 return c.Redirect(Menu1Controller.List)
732 return c.Redirect(pesticide_infoController.List)
733 }
734 //Excel 数据导出
735 func Excelout() {
736     // 创建一个新的 Excel 文件
737     file := excelize.NewFile()
738     // 定义数据
739     employees := []Employee{
740         {1, "Alice", "Smith", 28, "Manager"},
741         {2, "Bob", "Johnson", 35, "Engineer"},
742         {3, "Charlie", "Wilson", 22, "Intern"},
743     }
744     // 创建第一个工作表
745     sheet1 := "Employees"
746     index1 := file.NewSheet(sheet1)
747     // 设置工作表的列标题
748     titles := map[string]string{
749         "A1": "ID",
750         "B1": "pesticide_id",
751         "C1": "pesticide_name",
752         "D1": "application_rate",
753         "E1": "dilution_ratio",
754     }
755     for cell, title := range titles {
756         file.SetCellValue(sheet1, cell, title)
757     }
758     // 写入数据到工作表
759     for i, emp := range employees {
760         rowIndex := i + 2 // 数据从第二行开始
761         file.SetCellValue(sheet1,
762             "A"+excelize.ToAlphaString(1)+excelize.ToSliceIndex(rowIndex), emp.ID)
763         file.SetCellValue(sheet1,
764             "B"+excelize.ToAlphaString(2)+excelize.ToSliceIndex(rowIndex), emp.pesticide_id)
765         file.SetCellValue(sheet1,

```

```

766 "C"+excelize.ToAlphaString(3)+excelize.ToSliceIndex(rowIndex), emp.pesticide_name)
767     file.SetCellValue(sheet1,
768 "D"+excelize.ToAlphaString(4)+excelize.ToSliceIndex(rowIndex),
769 emp.application_rate)
770     file.SetCellValue(sheet1,
771 "E"+excelize.ToAlphaString(5)+excelize.ToSliceIndex(rowIndex), emp.dilution_ratio)
772 file.SetCellValue(sheet1,
773 "E"+excelize.ToAlphaString(6)+excelize.ToSliceIndex(rowIndex), emp.admin)
774 }
775 // 自动调整列宽
776 for col := 'A'; col <= 'E'; col++ {
777     colStr := string(col)
778     file.SetColWidth(sheet1, colStr, colStr, 15)
779 }
780 // 设置第一个工作表为活动工作表
781 file.SetActiveSheet(index1)
782 // 创建第二个工作表
783 sheet2 := "MoreData"
784 index2 := file.NewSheet(sheet2)
785
786 // 保存文件
787 filePath := "output.xlsx"
788 if err := file.SaveAs(filePath); err != nil {
789     log.Fatalf("Save Excel file error: %v", err)
790 }
791 // 提示文件已保存
792 log.Printf("Excel file '%s' has been saved successfully.\n", filePath)
793 }
794 // FDF 导出
795 func (c pesticide_infoController) ExportFDF() revel.Result {
796     var pesticide_info []pesticide_info
797     c.DB.Find(&pesticide_info)
798     // TODO: 实现 FDF 导出逻辑
799     var pesticide_info []Menu1
800     if err := c.DB.Find(&pesticide_info).Error; err != nil {
801         revel.AppLog.Errorf("查询数据库失败: %v", err)
802         c.Flash.Error("查询数据库失败, 请稍后重试")
803         return c.Redirect(Menu1Controller.List)
804     }
805     // 创建 PDF 文档对象
806     pdf := gofpdf.New("P", "mm", "A4", "")
807     pdf.AddPage()
808     // 设置字体
809     pdf.SetFont("Arial", "", 12)
810     // 添加表头
811     pdf.CellFormat(10, 7, "序号", 1, 0, "C", false, 0, "")
812     pdf.CellFormat(30, 7, "pesticide_id", 1, 0, "C", false, 0, "")//农药编号
813     pdf.CellFormat(30, 7, "pesticide_name", 1, 0, "C", false, 0, "")//农药名称
814     pdf.CellFormat(30, 7, "application_rate", 1, 0, "C", false, 0, "")//施用比例(ml/
815 亩)
816     pdf.CellFormat(30, 7, "dilution_ratio", 1, 1, "C", false, 0, "")//稀释比例

```



```

817 // 添加数据行
818 for _, menu := range pesticide_info {
819     pdf.CellFormat(10, 7, strconv.Itoa(int(menu.序号)), 1, 0, "C", false, 0, "")
820     pdf.CellFormat(30, 7, menu.pesticide_id, 1, 0, "", false, 0, "")//农药编号
821     pdf.CellFormat(30, 7, menu.pesticide_name, 1, 0, "", false, 0, "")//农药名称
822     pdf.CellFormat(30, 7, menu.application_rate, 1, 0, "", false, 0, "")//施用比
823 例(ml/亩)
824     pdf.CellFormat(30, 7, menu.dilution_ratio, 1, 1, "", false, 0, "")//稀释比
825 例    }
826 // 保存 PDF 文件
827 filePath := "path/to/save/yourfile.pdf"
828 err := pdf.OutputFileAndClose(filePath)
829 if err != nil {
830     revel.AppLog.Errorf("保存 PDF 文件失败: %v", err)
831     c.Flash.Error("保存 PDF 文件失败, 请稍后重试")
832     return c.Redirect(MenuController.List)
833 }
834 // 返回 PDF 文件路径供下载
835 c.Response.ContentType = "application/pdf"
836 c.Response.Out.Header().Set("Content-Disposition", "attachment;
837 filename="+filePath)
838 return c.File(filePath)
839 return c.Render(pesticide_info)
840 }
841 package models
842 import "github.com/jinzhu/gorm"
843 // 定义菜单 1 数据模型
844 type pesticide_info struct {
845     gorm.Model
846     序号      int      `gorm:"column:序号;not null"`
847     pesticide_id  string `gorm:"column:pesticide_id;not null"`//农药编号
848     pesticide_name string `gorm:"column:pesticide_name;not null"`//农药名称
849     application_rate string `gorm:"column:application_rate"`//施用比例(ml/亩)
850     dilution_ratio string `gorm:"column:字段拼音 4"`//稀释比例
851     操作者      string `gorm:"column:操作者"`
852 }
853 func init() {
854     // 路由映射
855     revel.Router.
856         Route("/pesticide_info/list/{page}",
857             &controllers.pesticide_infoController{}).
858         Route("/pesticide_info/add", &controllers.pesticide_infoController{}).
859         Route("/pesticide_info/edit", &controllers.pesticide_infoController{}).
860         Route("/pesticide_info/delete/{id}",
861             &controllers.pesticide_infoController{}).
862         Route("/pesticide_info/search/{keyword}/{page}",
863             &controllers.pesticide_infoController{}).
864         Route("/pesticide_info/import", &controllers.pesticide_infoController{}).
865         Route("/pesticide_info/export", &controllers.pesticide_infoController{}).
866     }
867 //油菜生长阶段监测模块的 go lang 代码

```

```

868 package controllers
869 import (
870     "log"
871     "os"
872     "github.com/revel/revel"
873     "github.com/360EntSecGroup-Skylar/excelize/v2"
874     "github.com/jinzhu/gorm"
875     "github.com/jinzhu/gorm/dialects/mysql"
876     "github.com/revel/revel"
877     "github.com/jinzhu/gorm"
878     "github.com/jung-kurt/gofpdf"
879     "your-app-name/app/controllers"
880     "github.com/revel/revel"
881     _ "github.com/jinzhu/gorm/dialects/postgres"
882     "github.com/jinzhu/gorm"
883 )
884 func InitDB() {
885     var err error
886     // Read database configuration from Revel configuration
887     dbConfig := revel.Config.StringDefault("db.url",
888 "postgres://localhost:5432/mydatabase")
889     // Open a connection to the database
890     DB, err = gorm.Open("postgres", dbConfig)
891     if err != nil {
892         revel.AppLog.Fatalf("Failed to connect to database: %v", err)
893     }
894     // Enable logging if needed
895     DB.LogMode(true)
896 }
897 func CloseDB() {
898     if DB != nil {
899         if err := DB.Close(); err != nil {
900             revel.AppLog.Errorf("Failed to close database connection: %v", err)
901         }
902     }
903 }
904 // 油菜生长阶段监测管理控制器
905 type growth_stageController struct {
906     *revel.Controller
907     DB *gorm.DB // 数据库连接对象
908 }
909 // 初始化数据库连接
910 func (c growth_stageController) Begin() revel.Result {
911     db, err := gorm.Open("mysql",
912 "user:password@/dbname?charset=utf8&parseTime=True&loc=Local")
913     if err != nil {
914         revel.AppLog.Error("Failed to connect database")
915         panic(err)
916     }
917     c.DB = db
918     return nil

```

```

919 }
920 // 结束时关闭数据库连接
921 func (c growth_stageController) End() revel.Result {
922     err := c.DB.Close()
923     if err != nil {
924         revel.AppLog.Error("Failed to close database connection")
925     }
926     return nil
927 }
928 // 列表展示
929 func (c growth_stageController) List(page int) revel.Result {
930     var growth_stage []growth_stage
931     c.DB.Offset((page - 1) * PageSize).Limit(PageSize).Find(&growth_stage)
932     return c.Render(growth_stage)
933 }
934 // 数据添加
935 func (c growth_stageController) Add(menu growth_stage) revel.Result {
936     c.Validation.Required(menu.Field1).Message("阶段编号不能为空")
937     c.Validation.Required(menu.Field2).Message("地块编号不能为空")
938     if c.Validation.HasErrors() {
939         c.Validation.Keep()
940         c.FlashParams()
941         return c.Redirect(growth_stageController.List)
942     }
943     c.DB.Create(&menu)
944     return c.Redirect(growth_stageController.List)
945 }
946 // 数据编辑
947 func (c growth_stageController) Edit(menu growth_stage) revel.Result {
948     c.DB.Save(&menu)
949     return c.Redirect(growth_stageController.List)
950 }
951 // 数据删除
952 func (c growth_stageController) Delete(id int) revel.Result {
953     var menu growth_stage
954     c.DB.First(&menu, id)
955     c.DB.Delete(&menu)
956     return c.Redirect(growth_stageController.List)
957 }
958 // 数据分页
959 const PageSize = 10
960 // 数据查询
961 func (c growth_stageController) Search(keyword string, page int) revel.Result {
962     var growth_stage []growth_stage
963     c.DB.Where("字段 1 LIKE ?", "%"+keyword+"%").Offset((page - 1) *
964     PageSize).Limit(PageSize).Find(&growth_stage)
965     return c.Render(growth_stage)
966 }
967 // Excel 导入
968 func (c growth_stageController) ImportExcel(file revel.File) revel.Result {
969     f, err := excelize.OpenReader(file)

```

```

970     if err != nil {
971         revel.AppLog.Error("Failed to open Excel file")
972         return c.Redirect(growth_stageController.List)
973     }
974     // 读取并处理 Excel 数据
975     // TODO: 实现 Excel 数据处理逻辑
976     if file == nil {
977         c.Flash.Error("请选择要导入的 Excel 文件")
978         return c.Redirect(Menu1Controller.List)
979     }
980     // 打开 Excel 文件
981     f, err := excelize.OpenReader(file)
982     if err != nil {
983         revel.AppLog.Errorf("打开 Excel 文件失败: %v", err)
984         c.Flash.Error("打开 Excel 文件失败, 请检查文件格式")
985         return c.Redirect(Menu1Controller.List)
986     }
987     // 获取工作表名称
988     sheetName := f.GetSheetName(1) // 假设只有一个工作表, 索引从 1 开始
989     rows, err := f.GetRows(sheetName)
990     if err != nil {
991         revel.AppLog.Errorf("获取 Excel 工作表行数据失败: %v", err)
992         c.Flash.Error("获取 Excel 工作表行数据失败, 请检查文件内容")
993         return c.Redirect(Menu1Controller.List)
994     }
995     var growth_stage []Menu1
996     for rowIndex, row := range rows {
997         if rowIndex == 0 {
998             continue // 跳过表头
999         }
1000         if len(row) < 5 {
1001             revel.AppLog.Errorf("Excel 行数据格式不正确: %v", row)
1002             continue // 跳过格式不正确的行
1003         }
1004         // 解析 Excel 数据
1005         menu := Menu1{
1006             序号:    rowIndex, // 假设序号是自动生成的
1007             stage_id: row[0], // 阶段编号
1008             field_id: row[1], // 地块编号
1009             growth_stage: row[2], // 生长阶段
1010             observation_date: row[3], // 观测日期
1011             操作者: "admin", // 假设当前操作者是管理员
1012         }
1013         // 将数据添加到列表
1014         growth_stage = append(growth_stage, menu)
1015     }
1016     // 将数据批量插入数据库
1017     if err := c.DB.Create(&growth_stage).Error; err != nil {
1018         revel.AppLog.Errorf("批量插入 Excel 数据到数据库失败: %v", err)
1019         c.Flash.Error("导入 Excel 数据到数据库失败, 请稍后重试")
1020         return c.Redirect(Menu1Controller.List)

```

```

1021     }
1022     c.Flash.Success("成功导入 Excel 数据")
1023     return c.Redirect(MenuController.List)
1024     return c.Redirect(growth_stageController.List)
1025 }
1026 //Excel 数据导出
1027 func Excelout() {
1028     // 创建一个新的 Excel 文件
1029     file := excelize.NewFile()
1030     // 定义数据
1031     employees := []Employee{
1032         {1, "Alice", "Smith", 28, "Manager"},
1033         {2, "Bob", "Johnson", 35, "Engineer"},
1034         {3, "Charlie", "Wilson", 22, "Intern"},
1035     }
1036     // 创建第一个工作表
1037     sheet1 := "Employees"
1038     index1 := file.NewSheet(sheet1)
1039     // 设置工作表的列标题
1040     titles := map[string]string{
1041         "A1": "ID",
1042         "B1": "stage_id",
1043         "C1": "field_id",
1044         "D1": "growth_stage",
1045         "E1": "observation_date",
1046     }
1047     for cell, title := range titles {
1048         file.SetCellValue(sheet1, cell, title)
1049     }
1050     // 写入数据到工作表
1051     for i, emp := range employees {
1052         rowIndex := i + 2 // 数据从第二行开始
1053         file.SetCellValue(sheet1,
1054             "A"+excelize.ToAlphaString(1)+excelize.ToSliceIndex(rowIndex), emp.ID)
1055         file.SetCellValue(sheet1,
1056             "B"+excelize.ToAlphaString(2)+excelize.ToSliceIndex(rowIndex), emp.stage_id)
1057         file.SetCellValue(sheet1,
1058             "C"+excelize.ToAlphaString(3)+excelize.ToSliceIndex(rowIndex), emp.field_id)
1059         file.SetCellValue(sheet1,
1060             "D"+excelize.ToAlphaString(4)+excelize.ToSliceIndex(rowIndex), emp.growth_stage)
1061         file.SetCellValue(sheet1,
1062             "E"+excelize.ToAlphaString(5)+excelize.ToSliceIndex(rowIndex),
1063             emp.observation_date)
1064         file.SetCellValue(sheet1,
1065             "E"+excelize.ToAlphaString(6)+excelize.ToSliceIndex(rowIndex), emp.admin)
1066     }
1067     // 自动调整列宽
1068     for col := 'A'; col <= 'E'; col++ {
1069         colStr := string(col)
1070         file.SetColWidth(sheet1, colStr, colStr, 15)
1071     }

```

```

1072 // 设置第一个工作表为活动工作表
1073 file.SetActiveSheet(index1)
1074 // 创建第二个工作表
1075 sheet2 := "MoreData"
1076 index2 := file.NewSheet(sheet2)
1077
1078 // 保存文件
1079 filePath := "output.xlsx"
1080 if err := file.SaveAs(filePath); err != nil {
1081     log.Fatalf("Save Excel file error: %v", err)
1082 }
1083 // 提示文件已保存
1084 log.Printf("Excel file '%s' has been saved successfully.\n", filePath)
1085 }
1086 // FDF 导出
1087 func (c growth_stageController) ExportFDF() revel.Result {
1088     var growth_stage []growth_stage
1089     c.DB.Find(&growth_stage)
1090     // TODO: 实现 FDF 导出逻辑
1091     var growth_stage []Menu1
1092     if err := c.DB.Find(&growth_stage).Error; err != nil {
1093         revel.AppLog.Errorf("查询数据库失败: %v", err)
1094         c.Flash.Error("查询数据库失败, 请稍后重试")
1095         return c.Redirect(Menu1Controller.List)
1096     }
1097     // 创建 PDF 文档对象
1098     pdf := gofpdf.New("P", "mm", "A4", "")
1099     pdf.AddPage()
1100     // 设置字体
1101     pdf.SetFont("Arial", "", 12)
1102     // 添加表头
1103     pdf.CellFormat(10, 7, "序号", 1, 0, "C", false, 0, "")
1104     pdf.CellFormat(30, 7, "stage_id", 1, 0, "C", false, 0, "")//阶段编号
1105     pdf.CellFormat(30, 7, "field_id", 1, 0, "C", false, 0, "")//地块编号
1106     pdf.CellFormat(30, 7, "growth_stage", 1, 0, "C", false, 0, "")//生长阶段
1107     pdf.CellFormat(30, 7, "observation_date", 1, 1, "C", false, 0, "")//观测日期
1108     // 添加数据行
1109     for _, menu := range growth_stage {
1110         pdf.CellFormat(10, 7, strconv.Itoa(int(menu.序号)), 1, 0, "C", false, 0, "")
1111         pdf.CellFormat(30, 7, menu.stage_id, 1, 0, "", false, 0, "")//阶段编号
1112         pdf.CellFormat(30, 7, menu.field_id, 1, 0, "", false, 0, "")//地块编号
1113         pdf.CellFormat(30, 7, menu.growth_stage, 1, 0, "", false, 0, "")//生长阶段
1114         pdf.CellFormat(30, 7, menu.observation_date, 1, 1, "", false, 0, "")//观测日
1115     }
1116     // 保存 PDF 文件
1117     filePath := "path/to/save/yourfile.pdf"
1118     err := pdf.OutputFileAndClose(filePath)
1119     if err != nil {
1120         revel.AppLog.Errorf("保存 PDF 文件失败: %v", err)
1121         c.Flash.Error("保存 PDF 文件失败, 请稍后重试")
1122         return c.Redirect(Menu1Controller.List)

```

```

1123     }
1124     // 返回 PDF 文件路径供下载
1125     c.Response.ContentType = "application/pdf"
1126     c.Response.Out.Header().Set("Content-Disposition", "attachment;
1127 filename="+filePath)
1128     return c.File(filePath)
1129     return c.Render(growth_stage)
1130 }
1131 package models
1132 import "github.com/jinzhu/gorm"
1133 // 定义菜单 1 数据模型
1134 type growth_stage struct {
1135     gorm.Model
1136     序号      int      `gorm:"column:序号;not null"`
1137     stage_id   string `gorm:"column:stage_id;not null"`//阶段编号
1138     field_id   string `gorm:"column:field_id;not null"`//地块编号
1139     growth_stage string `gorm:"column:growth_stage"`//生长阶段
1140     observation_date string `gorm:"column:字段拼音 4"`//观测日期
1141     操作者     string `gorm:"column:操作者"`
1142 }
1143 func init() {
1144     // 路由映射
1145     revel.Router.
1146         Route("/growth_stage/list/{page}", &controllers.growth_stageController{}).
1147         Route("/growth_stage/add", &controllers.growth_stageController{}).
1148         Route("/growth_stage/edit", &controllers.growth_stageController{}).
1149         Route("/growth_stage/delete/{id}", &controllers.growth_stageController{}).
1150         Route("/growth_stage/search/{keyword}/{page}",
1151 &controllers.growth_stageController{}).
1152         Route("/growth_stage/import", &controllers.growth_stageController{}).
1153         Route("/growth_stage/export", &controllers.growth_stageController{}).
1154 }
1155 //病虫害识别记录模块的 go 语言代码
1156 package controllers
1157 import (
1158     "log"
1159     "os"
1160     "github.com/revel/revel"
1161     "github.com/360EntSecGroup-Skylar/excelize/v2"
1162     "github.com/jinzhu/gorm"
1163     "github.com/jinzhu/gorm/dialects/mysql"
1164     "github.com/revel/revel"
1165     "github.com/jinzhu/gorm"
1166     "github.com/jung-kurt/gofpdf"
1167     "your-app-name/app/controllers"
1168     "github.com/revel/revel"
1169     _ "github.com/jinzhu/gorm/dialects/postgres"
1170     "github.com/jinzhu/gorm"
1171 )
1172 func InitDB() {
1173     var err error

```

```
1174 // Read database configuration from Revel configuration
1175 dbConfig := revel.Config.StringDefault("db.url",
1176 "postgres://localhost:5432/mydatabase")
1177 // Open a connection to the database
1178 DB, err = gorm.Open("postgres", dbConfig)
1179 if err != nil {
1180     revel.AppLog.Fatalf("Failed to connect to database: %v", err)
1181 }
1182 // Enable logging if needed
1183 DB.LogMode(true)
1184 }
1185 func CloseDB() {
1186     if DB != nil {
1187         if err := DB.Close(); err != nil {
1188             revel.AppLog.Errorf("Failed to close database connection: %v", err)
1189         }
1190     }
1191 }
1192 // 病虫害识别记录管理控制器
1193 type pest_diseaseController struct {
1194     *revel.Controller
1195     DB *gorm.DB // 数据库连接对象
1196 }
1197 // 初始化数据库连接
1198 func (c pest_diseaseController) Begin() revel.Result {
1199     db, err := gorm.Open("mysql",
1200 "user:password@/dbname?charset=utf8&parseTime=True&loc=Local")
1201     if err != nil {
1202         revel.AppLog.Error("Failed to connect database")
1203         panic(err)
1204     }
1205     c.DB = db
1206     return nil
1207 }
1208 // 结束时关闭数据库连接
1209 func (c pest_diseaseController) End() revel.Result {
1210     err := c.DB.Close()
1211     if err != nil {
1212         revel.AppLog.Error("Failed to close database connection")
1213     }
1214     return nil
1215 }
1216 // 列表展示
1217 func (c pest_diseaseController) List(page int) revel.Result {
1218     var pest_disease []pest_disease
1219     c.DB.Offset((page - 1) * PageSize).Limit(PageSize).Find(&pest_disease)
1220     return c.Render(pest_disease)
1221 }
1222 // 数据添加
1223 func (c pest_diseaseController) Add(menu pest_disease) revel.Result {
1224     c.Validation.Required(menu.Field1).Message("病害编号不能为空")
1225 }
```



```

1225     c.Validation.Required(menu.Field2).Message("地块编号不能为空")
1226     if c.Validation.HasErrors() {
1227         c.Validation.Keep()
1228         c.FlashParams()
1229         return c.Redirect(pest_diseaseController.List)
1230     }
1231     c.DB.Create(&menu)
1232     return c.Redirect(pest_diseaseController.List)
1233 }
1234 // 数据编辑
1235 func (c pest_diseaseController) Edit(menu pest_disease) revel.Result {
1236     c.DB.Save(&menu)
1237     return c.Redirect(pest_diseaseController.List)
1238 }
1239 // 数据删除
1240 func (c pest_diseaseController) Delete(id int) revel.Result {
1241     var menu pest_disease
1242     c.DB.First(&menu, id)
1243     c.DB.Delete(&menu)
1244     return c.Redirect(pest_diseaseController.List)
1245 }
1246 // 数据分页
1247 const PageSize = 10
1248 // 数据查询
1249 func (c pest_diseaseController) Search(keyword string, page int) revel.Result {
1250     var pest_disease []pest_disease
1251     c.DB.Where(" 字段 1 LIKE ?", "%"+keyword+"%").Offset((page - 1) *
1252     PageSize).Limit(PageSize).Find(&pest_disease)
1253     return c.Render(pest_disease)
1254 }
1255 // Excel 导入
1256 func (c pest_diseaseController) ImportExcel(file revel.File) revel.Result {
1257     f, err := excelize.OpenReader(file)
1258     if err != nil {
1259         revel.AppLog.Error("Failed to open Excel file")
1260         return c.Redirect(pest_diseaseController.List)
1261     }
1262     // 读取并处理 Excel 数据
1263     // TODO: 实现 Excel 数据处理逻辑
1264     if file == nil {
1265         c.Flash.Error("请选择要导入的 Excel 文件")
1266         return c.Redirect(MenuController.List)
1267     }
1268     // 打开 Excel 文件
1269     f, err := excelize.OpenReader(file)
1270     if err != nil {
1271         revel.AppLog.Errorf("打开 Excel 文件失败: %v", err)
1272         c.Flash.Error("打开 Excel 文件失败, 请检查文件格式")
1273         return c.Redirect(MenuController.List)
1274     }
1275     // 获取工作表名称

```

```

1276 sheetName := f.GetSheetName(1) // 假设只有一个工作表, 索引从 1 开始
1277 rows, err := f.GetRows(sheetName)
1278 if err != nil {
1279     revel.AppLog.Errorf("获取 Excel 工作表行数据失败: %v", err)
1280     c.Flash.Error("获取 Excel 工作表行数据失败, 请检查文件内容")
1281     return c.Redirect(MenuController.List)
1282 }
1283 var pest_disease []Menu1
1284 for rowIndex, row := range rows {
1285     if rowIndex == 0 {
1286         continue // 跳过表头
1287     }
1288     if len(row) < 5 {
1289         revel.AppLog.Errorf("Excel 行数据格式不正确: %v", row)
1290         continue // 跳过格式不正确的行
1291     }
1292     // 解析 Excel 数据
1293     menu := Menu1{
1294         序号:    rowIndex, // 假设序号是自动生成的
1295         disease_id: row[0], // 病害编号
1296         field_id: row[1], // 地块编号
1297         disease_name: row[2], // 病害名称
1298         severity_level: row[3], // 严重程度
1299         操作者: "admin", // 假设当前操作者是管理员
1300     }
1301     // 将数据添加到列表
1302     pest_disease = append(pest_disease, menu)
1303 }
1304 // 将数据批量插入数据库
1305 if err := c.DB.Create(&pest_disease).Error; err != nil {
1306     revel.AppLog.Errorf("批量插入 Excel 数据到数据库失败: %v", err)
1307     c.Flash.Error("导入 Excel 数据到数据库失败, 请稍后重试")
1308     return c.Redirect(MenuController.List)
1309 }
1310 c.Flash.Success("成功导入 Excel 数据")
1311 return c.Redirect(MenuController.List)
1312 return c.Redirect(pest_diseaseController.List)
1313 }
1314 //Excel 数据导出
1315 func Excelout() {
1316     // 创建一个新的 Excel 文件
1317     file := excelize.NewFile()
1318     // 定义数据
1319     employees := []Employee{
1320         {1, "Alice", "Smith", 28, "Manager"},
1321         {2, "Bob", "Johnson", 35, "Engineer"},
1322         {3, "Charlie", "Wilson", 22, "Intern"},
1323     }
1324     // 创建第一个工作表
1325     sheet1 := "Employees"
1326     index1 := file.NewSheet(sheet1)

```

```

1327 // 设置工作表的列标题
1328 titles := map[string]string{
1329     "A1": "ID",
1330     "B1": "disease_id",
1331     "C1": "field_id",
1332     "D1": "disease_name",
1333     "E1": "severity_level",
1334 }
1335 for cell, title := range titles {
1336     file.SetCellValue(sheet1, cell, title)
1337 }
1338 // 写入数据到工作表
1339 for i, emp := range employees {
1340     rowIndex := i + 2 // 数据从第二行开始
1341     file.SetCellValue(sheet1,
1342 "A"+excelize.ToAlphaString(1)+excelize.ToSliceIndex(rowIndex), emp.ID)
1343     file.SetCellValue(sheet1,
1344 "B"+excelize.ToAlphaString(2)+excelize.ToSliceIndex(rowIndex), emp.disease_id)
1345     file.SetCellValue(sheet1,
1346 "C"+excelize.ToAlphaString(3)+excelize.ToSliceIndex(rowIndex), emp.field_id)
1347     file.SetCellValue(sheet1,
1348 "D"+excelize.ToAlphaString(4)+excelize.ToSliceIndex(rowIndex), emp.disease_name)
1349     file.SetCellValue(sheet1,
1350 "E"+excelize.ToAlphaString(5)+excelize.ToSliceIndex(rowIndex), emp.severity_level)
1351 file.SetCellValue(sheet1,
1352 "E"+excelize.ToAlphaString(6)+excelize.ToSliceIndex(rowIndex), emp.admin)
1353 }
1354 // 自动调整列宽
1355 for col := 'A'; col <= 'E'; col++ {
1356     colStr := string(col)
1357     file.SetColWidth(sheet1, colStr, colStr, 15)
1358 }
1359 // 设置第一个工作表为活动工作表
1360 file.SetActiveSheet(index1)
1361 // 创建第二个工作表
1362 sheet2 := "MoreData"
1363 index2 := file.NewSheet(sheet2)
1364
1365 // 保存文件
1366 filePath := "output.xlsx"
1367 if err := file.SaveAs(filePath); err != nil {
1368     log.Fatalf("Save Excel file error: %v", err)
1369 }
1370 // 提示文件已保存
1371 log.Printf("Excel file '%s' has been saved successfully.\n", filePath)
1372 }
1373 // FDF 导出
1374 func (c pest_diseaseController) ExportFDF() revel.Result {
1375     var pest_disease []pest_disease
1376     c.DB.Find(&pest_disease)
1377     // TODO: 实现 FDF 导出逻辑

```

```

1378     var pest_disease []Menu1
1379     if err := c.DB.Find(&pest_disease).Error; err != nil {
1380         revel.AppLog.Errorf("查询数据库失败: %v", err)
1381         c.Flash.Error("查询数据库失败, 请稍后重试")
1382         return c.Redirect(Menu1Controller.List)
1383     }
1384     // 创建 PDF 文档对象
1385     pdf := gofpdf.New("P", "mm", "A4", "")
1386     pdf.AddPage()
1387     // 设置字体
1388     pdf.SetFont("Arial", "", 12)
1389     // 添加表头
1390     pdf.CellFormat(10, 7, "序号", 1, 0, "C", false, 0, "")
1391     pdf.CellFormat(30, 7, "disease_id", 1, 0, "C", false, 0, "")//病害编号
1392     pdf.CellFormat(30, 7, "field_id", 1, 0, "C", false, 0, "")//地块编号
1393     pdf.CellFormat(30, 7, "disease_name", 1, 0, "C", false, 0, "")//病害名称
1394     pdf.CellFormat(30, 7, "severity_level", 1, 1, "C", false, 0, "")//严重程度
1395     // 添加数据行
1396     for _, menu := range pest_disease {
1397         pdf.CellFormat(10, 7, strconv.Itoa(int(menu.序号)), 1, 0, "C", false, 0, "")
1398         pdf.CellFormat(30, 7, menu.disease_id, 1, 0, "", false, 0, "")//病害编号
1399         pdf.CellFormat(30, 7, menu.field_id, 1, 0, "", false, 0, "")//地块编号
1400         pdf.CellFormat(30, 7, menu.disease_name, 1, 0, "", false, 0, "")//病害名称
1401         pdf.CellFormat(30, 7, menu.severity_level, 1, 1, "", false, 0, "")//严重程
1402     度    }
1403     // 保存 PDF 文件
1404     filePath := "path/to/save/yourfile.pdf"
1405     err := pdf.OutputFileAndClose(filePath)
1406     if err != nil {
1407         revel.AppLog.Errorf("保存 PDF 文件失败: %v", err)
1408         c.Flash.Error("保存 PDF 文件失败, 请稍后重试")
1409         return c.Redirect(Menu1Controller.List)
1410     }
1411     // 返回 PDF 文件路径供下载
1412     c.Response.ContentType = "application/pdf"
1413     c.Response.Out.Header().Set("Content-Disposition", "attachment;
1414 filename="+filePath)
1415     return c.File(filePath)
1416     return c.Render(pest_disease)
1417 }
1418 package models
1419 import "github.com/jinzhu/gorm"
1420 // 定义菜单 1 数据模型
1421 type pest_disease struct {
1422     gorm.Model
1423     序号      int      `gorm:"column:序号;not null"`
1424     disease_id string `gorm:"column:disease_id;not null"`//病害编号
1425     field_id  string `gorm:"column:field_id;not null"`//地块编号
1426     disease_name string `gorm:"column:disease_name"`//病害名称
1427     severity_level string `gorm:"column:字段拼音 4"`//严重程度
1428     操作者    string `gorm:"column:操作者"`

```

```

1429 }
1430 func init() {
1431     // 路由映射
1432     revel.Router.
1433         Route("/pest_disease/list/{page}", &controllers.pest_diseaseController{}).
1434         Route("/pest_disease/add", &controllers.pest_diseaseController{}).
1435         Route("/pest_disease/edit", &controllers.pest_diseaseController{}).
1436         Route("/pest_disease/delete/{id}", &controllers.pest_diseaseController{}).
1437         Route("/pest_disease/search/{keyword}/{page}",
1438 &controllers.pest_diseaseController{}).
1439         Route("/pest_disease/import", &controllers.pest_diseaseController{}).
1440         Route("/pest_disease/export", &controllers.pest_diseaseController{}).
1441 }
1442 //飞行路径规划记录模块的 go lang 代码
1443 package controllers
1444 import (
1445     "log"
1446     "os"
1447     "github.com/revel/revel"
1448     "github.com/360EntSecGroup-Skylar/excelize/v2"
1449     "github.com/jinzhu/gorm"
1450     "github.com/jinzhu/gorm/dialects/mysql"
1451     "github.com/revel/revel"
1452     "github.com/jinzhu/gorm"
1453     "github.com/jung-kurt/gofpdf"
1454     "your-app-name/app/controllers"
1455     "github.com/revel/revel"
1456     _ "github.com/jinzhu/gorm/dialects/postgres"
1457     "github.com/jinzhu/gorm"
1458 )
1459 func InitDB() {
1460     var err error
1461     // Read database configuration from Revel configuration
1462     dbConfig := revel.Config.StringDefault("db.url",
1463 "postgres://localhost:5432/mydatabase")
1464     // Open a connection to the database
1465     DB, err = gorm.Open("postgres", dbConfig)
1466     if err != nil {
1467         revel.AppLog.Fatalf("Failed to connect to database: %v", err)
1468     }
1469     // Enable logging if needed
1470     DB.LogMode(true)
1471 }
1472 func CloseDB() {
1473     if DB != nil {
1474         if err := DB.Close(); err != nil {
1475             revel.AppLog.Errorf("Failed to close database connection: %v", err)
1476         }
1477     }
1478 }
1479 // 飞行路径规划记录管理控制器

```

```

1480 type flight_pathController struct {
1481     *revel.Controller
1482     DB *gorm.DB // 数据库连接对象
1483 }
1484 // 初始化数据库连接
1485 func (c flight_pathController) Begin() revel.Result {
1486     db, err := gorm.Open("mysql",
1487         "user:password@/dbname?charset=utf8&parseTime=True&loc=Local")
1488     if err != nil {
1489         revel.AppLog.Error("Failed to connect database")
1490         panic(err)
1491     }
1492     c.DB = db
1493     return nil
1494 }
1495 // 结束时关闭数据库连接
1496 func (c flight_pathController) End() revel.Result {
1497     err := c.DB.Close()
1498     if err != nil {
1499         revel.AppLog.Error("Failed to close database connection")
1500     }
1501     return nil
1502 }
1503 // 列表展示
1504 func (c flight_pathController) List(page int) revel.Result {
1505     var flight_path []flight_path
1506     c.DB.Offset((page - 1) * PageSize).Limit(PageSize).Find(&flight_path)
1507     return c.Render(flight_path)
1508 }
1509 // 数据添加
1510 func (c flight_pathController) Add(menu flight_path) revel.Result {
1511     c.Validation.Required(menu.Field1).Message("路径编号不能为空")
1512     c.Validation.Required(menu.Field2).Message("地块编号不能为空")
1513     if c.Validation.HasErrors() {
1514         c.Validation.Keep()
1515         c.FlashParams()
1516         return c.Redirect(flight_pathController.List)
1517     }
1518     c.DB.Create(&menu)
1519     return c.Redirect(flight_pathController.List)
1520 }
1521 // 数据编辑
1522 func (c flight_pathController) Edit(menu flight_path) revel.Result {
1523     c.DB.Save(&menu)
1524     return c.Redirect(flight_pathController.List)
1525 }
1526 // 数据删除
1527 func (c flight_pathController) Delete(id int) revel.Result {
1528     var menu flight_path
1529     c.DB.First(&menu, id)
1530     c.DB.Delete(&menu)

```

```

1531     return c.Redirect(flight_pathController.List)
1532 }
1533 // 数据分页
1534 const PageSize = 10
1535 // 数据查询
1536 func (c flight_pathController) Search(keyword string, page int) revel.Result {
1537     var flight_path []flight_path
1538     c.DB.Where(" 字 段 1 LIKE ?", "%"+keyword+"%").Offset((page - 1) *
1539     PageSize).Limit(PageSize).Find(&flight_path)
1540     return c.Render(flight_path)
1541 }
1542 // Excel 导入
1543 func (c flight_pathController) ImportExcel(file revel.File) revel.Result {
1544     f, err := excelize.OpenReader(file)
1545     if err != nil {
1546         revel.AppLog.Error("Failed to open Excel file")
1547         return c.Redirect(flight_pathController.List)
1548     }
1549     // 读取并处理 Excel 数据
1550     // TODO: 实现 Excel 数据处理逻辑
1551     if file == nil {
1552         c.Flash.Error("请选择要导入的 Excel 文件")
1553         return c.Redirect(Menu1Controller.List)
1554     }
1555     // 打开 Excel 文件
1556     f, err := excelize.OpenReader(file)
1557     if err != nil {
1558         revel.AppLog.Errorf("打开 Excel 文件失败: %v", err)
1559         c.Flash.Error("打开 Excel 文件失败, 请检查文件格式")
1560         return c.Redirect(Menu1Controller.List)
1561     }
1562     // 获取工作表名称
1563     sheetName := f.GetSheetName(1) // 假设只有一个工作表, 索引从 1 开始
1564     rows, err := f.GetRows(sheetName)
1565     if err != nil {
1566         revel.AppLog.Errorf("获取 Excel 工作表行数据失败: %v", err)
1567         c.Flash.Error("获取 Excel 工作表行数据失败, 请检查文件内容")
1568         return c.Redirect(Menu1Controller.List)
1569     }
1570     var flight_path []Menu1
1571     for rowIndex, row := range rows {
1572         if rowIndex == 0 {
1573             continue // 跳过表头
1574         }
1575         if len(row) < 5 {
1576             revel.AppLog.Errorf("Excel 行数据格式不正确: %v", row)
1577             continue // 跳过格式不正确的行
1578         }
1579         // 解析 Excel 数据
1580         menu := Menu1{
1581             序号:    rowIndex, // 假设序号是自动生成的

```

```

1582         path_id: row[0], //路径编号
1583         field_id: row[1], //地块编号
1584         path_coordinates: row[2], //路径坐标序列
1585         flight_altitude: row[3], //飞行高度(米)
1586         操作者: "admin", // 假设当前操作者是管理员
1587     }
1588     // 将数据添加到列表
1589     flight_path = append(flight_path, menu)
1590 }
1591 // 将数据批量插入数据库
1592 if err := c.DB.Create(&flight_path).Error; err != nil {
1593     revel.AppLog.Errorf("批量插入 Excel 数据到数据库失败: %v", err)
1594     c.Flash.Error("导入 Excel 数据到数据库失败, 请稍后重试")
1595     return c.Redirect(MenuController.List)
1596 }
1597 c.Flash.Success("成功导入 Excel 数据")
1598 return c.Redirect(MenuController.List)
1599 return c.Redirect(flight_pathController.List)
1600 }
1601 //Excel 数据导出
1602 func Excelout() {
1603     // 创建一个新的 Excel 文件
1604     file := excelize.NewFile()
1605     // 定义数据
1606     employees := []Employee{
1607         {1, "Alice", "Smith", 28, "Manager"},
1608         {2, "Bob", "Johnson", 35, "Engineer"},
1609         {3, "Charlie", "Wilson", 22, "Intern"},
1610     }
1611     // 创建第一个工作表
1612     sheet1 := "Employees"
1613     index1 := file.NewSheet(sheet1)
1614     // 设置工作表的列标题
1615     titles := map[string]string{
1616         "A1": "ID",
1617         "B1": "path_id",
1618         "C1": "field_id",
1619         "D1": "path_coordinates",
1620         "E1": "flight_altitude",
1621     }
1622     for cell, title := range titles {
1623         file.SetCellValue(sheet1, cell, title)
1624     }
1625     // 写入数据到工作表
1626     for i, emp := range employees {
1627         rowIndex := i + 2 // 数据从第二行开始
1628         file.SetCellValue(sheet1,
1629             "A"+excelize.ToAlphaString(1)+excelize.ToSliceIndex(rowIndex), emp.ID)
1630         file.SetCellValue(sheet1,
1631             "B"+excelize.ToAlphaString(2)+excelize.ToSliceIndex(rowIndex), emp.path_id)
1632         file.SetCellValue(sheet1,

```



```

1633 "C"+excelize.ToAlphaString(3)+excelize.ToSliceIndex(rowIndex), emp.field_id)
1634     file.SetCellValue(sheet1,
1635 "D"+excelize.ToAlphaString(4)+excelize.ToSliceIndex(rowIndex),
1636 emp.path_coordinates)
1637     file.SetCellValue(sheet1,
1638 "E"+excelize.ToAlphaString(5)+excelize.ToSliceIndex(rowIndex), emp.flight_altitude)
1639 file.SetCellValue(sheet1,
1640 "E"+excelize.ToAlphaString(6)+excelize.ToSliceIndex(rowIndex), emp.admin)
1641 }
1642 // 自动调整列宽
1643 for col := 'A'; col <= 'E'; col++ {
1644     colStr := string(col)
1645     file.SetColWidth(sheet1, colStr, colStr, 15)
1646 }
1647 // 设置第一个工作表为活动工作表
1648 file.SetActiveSheet(index1)
1649 // 创建第二个工作表
1650 sheet2 := "MoreData"
1651 index2 := file.NewSheet(sheet2)
1652
1653 // 保存文件
1654 filePath := "output.xlsx"
1655 if err := file.SaveAs(filePath); err != nil {
1656     log.Fatalf("Save Excel file error: %v", err)
1657 }
1658 // 提示文件已保存
1659 log.Printf("Excel file '%s' has been saved successfully.\n", filePath)
1660 }
1661 // FDF 导出
1662 func (c flight_pathController) ExportFDF() revel.Result {
1663     var flight_path []flight_path
1664     c.DB.Find(&flight_path)
1665     // TODO: 实现 FDF 导出逻辑
1666     var flight_path []Menu1
1667     if err := c.DB.Find(&flight_path).Error; err != nil {
1668         revel.AppLog.Errorf("查询数据库失败: %v", err)
1669         c.Flash.Error("查询数据库失败, 请稍后重试")
1670         return c.Redirect(Menu1Controller.List)
1671     }
1672     // 创建 PDF 文档对象
1673     pdf := gofpdf.New("P", "mm", "A4", "")
1674     pdf.AddPage()
1675     // 设置字体
1676     pdf.SetFont("Arial", "", 12)
1677     // 添加表头
1678     pdf.CellFormat(10, 7, "序号", 1, 0, "C", false, 0, "")
1679     pdf.CellFormat(30, 7, "path_id", 1, 0, "C", false, 0, "")//路径编号
1680     pdf.CellFormat(30, 7, "field_id", 1, 0, "C", false, 0, "")//地块编号
1681     pdf.CellFormat(30, 7, "path_coordinates", 1, 0, "C", false, 0, "")//路径坐标序列
1682     pdf.CellFormat(30, 7, "flight_altitude", 1, 1, "C", false, 0, "")//飞行高度(米)
1683     // 添加数据行

```

```

1684     for _, menu := range flight_path {
1685         pdf.CellFormat(10, 7, strconv.Itoa(int(menu.序号)), 1, 0, "C", false, 0, "")
1686         pdf.CellFormat(30, 7, menu.path_id, 1, 0, "", false, 0, "")//路径编号
1687         pdf.CellFormat(30, 7, menu.field_id, 1, 0, "", false, 0, "")//地块编号
1688         pdf.CellFormat(30, 7, menu.path_coordinates, 1, 0, "", false, 0, "")//路径坐
1689 标序列
1690         pdf.CellFormat(30, 7, menu.flight_altitude, 1, 1, "", false, 0, "")//飞行高
1691 度(米)    }
1692     // 保存 PDF 文件
1693     filePath := "path/to/save/yourfile.pdf"
1694     err := pdf.OutputFileAndClose(filePath)
1695     if err != nil {
1696         revel.AppLog.Errorf("保存 PDF 文件失败: %v", err)
1697         c.Flash.Error("保存 PDF 文件失败, 请稍后重试")
1698         return c.Redirect(MenuController.List)
1699     }
1700     // 返回 PDF 文件路径供下载
1701     c.Response.ContentType = "application/pdf"
1702     c.Response.Out.Header().Set("Content-Disposition", "attachment;
1703 filename="+filePath)
1704     return c.File(filePath)
1705     return c.Render(flight_path)
1706 }
1707 package models
1708 import "github.com/jinzhu/gorm"
1709 // 定义菜单 1 数据模型
1710 type flight_path struct {
1711     gorm.Model
1712     序号      int      `gorm:"column:序号;not null"`
1713     path_id    string  `gorm:"column:path_id;not null"`//路径编号
1714     field_id   string  `gorm:"column:field_id;not null"`//地块编号
1715     path_coordinates string `gorm:"column:path_coordinates"`//路径坐标序列
1716     flight_altitude string `gorm:"column:字段拼音 4"`//飞行高度(米)
1717     操作者     string  `gorm:"column:操作者"`
1718 }
1719 func init() {
1720     // 路由映射
1721     revel.Router.
1722         Route("/flight_path/list/{page}", &controllers.flight_pathController{}).
1723         Route("/flight_path/add", &controllers.flight_pathController{}).
1724         Route("/flight_path/edit", &controllers.flight_pathController{}).
1725         Route("/flight_path/delete/{id}", &controllers.flight_pathController{}).
1726         Route("/flight_path/search/{keyword}/{page}",
1727 &controllers.flight_pathController{}).
1728         Route("/flight_path/import", &controllers.flight_pathController{}).
1729         Route("/flight_path/export", &controllers.flight_pathController{}).
1730 }
1731 //喷洒作业执行记录模块的 go lang 代码
1732 package controllers
1733 import (
1734     "log"

```

```

1735     "os"
1736     "github.com/revel/revel"
1737     "github.com/360EntSecGroup-Skylar/excelize/v2"
1738     "github.com/jinzhu/gorm"
1739     "github.com/jinzhu/gorm/dialects/mysql"
1740     "github.com/revel/revel"
1741     "github.com/jinzhu/gorm"
1742     "github.com/jung-kurt/gofpdf"
1743     "your-app-name/app/controllers"
1744     "github.com/revel/revel"
1745     _ "github.com/jinzhu/gorm/dialects/postgres"
1746     "github.com/jinzhu/gorm"
1747 )
1748 func InitDB() {
1749     var err error
1750     // Read database configuration from Revel configuration
1751     dbConfig := revel.Config.StringDefault("db.url",
1752     "postgres://localhost:5432/mydatabase")
1753     // Open a connection to the database
1754     DB, err = gorm.Open("postgres", dbConfig)
1755     if err != nil {
1756         revel.AppLog.Fatalf("Failed to connect to database: %v", err)
1757     }
1758     // Enable logging if needed
1759     DB.LogMode(true)
1760 }
1761 func CloseDB() {
1762     if DB != nil {
1763         if err := DB.Close(); err != nil {
1764             revel.AppLog.Errorf("Failed to close database connection: %v", err)
1765         }
1766     }
1767 }
1768 // 喷洒作业执行记录管理控制器
1769 type spray_operationController struct {
1770     *revel.Controller
1771     DB *gorm.DB // 数据库连接对象
1772 }
1773 // 初始化数据库连接
1774 func (c spray_operationController) Begin() revel.Result {
1775     db, err := gorm.Open("mysql",
1776     "user:password@/dbname?charset=utf8&parseTime=True&loc=Local")
1777     if err != nil {
1778         revel.AppLog.Error("Failed to connect database")
1779         panic(err)
1780     }
1781     c.DB = db
1782     return nil
1783 }
1784 // 结束时关闭数据库连接
1785 func (c spray_operationController) End() revel.Result {

```

```

1786     err := c.DB.Close()
1787     if err != nil {
1788         revel.AppLog.Error("Failed to close database connection")
1789     }
1790     return nil
1791 }
1792 // 列表展示
1793 func (c spray_operationController) List(page int) revel.Result {
1794     var spray_operation []spray_operation
1795     c.DB.Offset((page - 1) * PageSize).Limit(PageSize).Find(&spray_operation)
1796     return c.Render(spray_operation)
1797 }
1798 // 数据添加
1799 func (c spray_operationController) Add(menu spray_operation) revel.Result {
1800     c.Validation.Required(menu.Field1).Message("作业编号不能为空")
1801     c.Validation.Required(menu.Field2).Message("无人机编号不能为空")
1802     if c.Validation.HasErrors() {
1803         c.Validation.Keep()
1804         c.FlashParams()
1805         return c.Redirect(spray_operationController.List)
1806     }
1807     c.DB.Create(&menu)
1808     return c.Redirect(spray_operationController.List)
1809 }
1810 // 数据编辑
1811 func (c spray_operationController) Edit(menu spray_operation) revel.Result {
1812     c.DB.Save(&menu)
1813     return c.Redirect(spray_operationController.List)
1814 }
1815 // 数据删除
1816 func (c spray_operationController) Delete(id int) revel.Result {
1817     var menu spray_operation
1818     c.DB.First(&menu, id)
1819     c.DB.Delete(&menu)
1820     return c.Redirect(spray_operationController.List)
1821 }
1822 // 数据分页
1823 const PageSize = 10
1824 // 数据查询
1825 func (c spray_operationController) Search(keyword string, page int) revel.Result {
1826     var spray_operation []spray_operation
1827     c.DB.Where(" 字 段 1 LIKE ?", "%"+keyword+"%").Offset((page - 1) *
1828     PageSize).Limit(PageSize).Find(&spray_operation)
1829     return c.Render(spray_operation)
1830 }
1831 // Excel 导入
1832 func (c spray_operationController) ImportExcel(file revel.File) revel.Result {
1833     f, err := excelize.OpenReader(file)
1834     if err != nil {
1835         revel.AppLog.Error("Failed to open Excel file")
1836         return c.Redirect(spray_operationController.List)

```

```

1837     }
1838     // 读取并处理 Excel 数据
1839     // TODO: 实现 Excel 数据处理逻辑
1840     if file == nil {
1841         c.Flash.Error("请选择要导入的 Excel 文件")
1842         return c.Redirect(MenuController.List)
1843     }
1844     // 打开 Excel 文件
1845     f, err := excelize.OpenReader(file)
1846     if err != nil {
1847         revel.AppLog.Errorf("打开 Excel 文件失败: %v", err)
1848         c.Flash.Error("打开 Excel 文件失败, 请检查文件格式")
1849         return c.Redirect(MenuController.List)
1850     }
1851     // 获取工作表名称
1852     sheetName := f.GetSheetName(1) // 假设只有一个工作表, 索引从 1 开始
1853     rows, err := f.GetRows(sheetName)
1854     if err != nil {
1855         revel.AppLog.Errorf("获取 Excel 工作表行数据失败: %v", err)
1856         c.Flash.Error("获取 Excel 工作表行数据失败, 请检查文件内容")
1857         return c.Redirect(MenuController.List)
1858     }
1859     var spray_operation []Menu
1860     for rowIndex, row := range rows {
1861         if rowIndex == 0 {
1862             continue // 跳过表头
1863         }
1864         if len(row) < 5 {
1865             revel.AppLog.Errorf("Excel 行数据格式不正确: %v", row)
1866             continue // 跳过格式不正确的行
1867         }
1868         // 解析 Excel 数据
1869         menu := Menu{
1870             序号:    rowIndex, // 假设序号是自动生成的
1871             operation_id: row[0], // 作业编号
1872             drone_id:   row[1], // 无人机编号
1873             field_id:   row[2], // 地块编号
1874             operation_time: row[3], // 作业时间
1875             操作者: "admin", // 假设当前操作者是管理员
1876         }
1877         // 将数据添加到列表
1878         spray_operation = append(spray_operation, menu)
1879     }
1880     // 将数据批量插入数据库
1881     if err := c.DB.Create(&spray_operation).Error; err != nil {
1882         revel.AppLog.Errorf("批量插入 Excel 数据到数据库失败: %v", err)
1883         c.Flash.Error("导入 Excel 数据到数据库失败, 请稍后重试")
1884         return c.Redirect(MenuController.List)
1885     }
1886     c.Flash.Success("成功导入 Excel 数据")
1887     return c.Redirect(MenuController.List)

```

```

1888     return c.Redirect(spray_operationController.List)
1889 }
1890 //Excel 数据导出
1891 func Excelout() {
1892     // 创建一个新的 Excel 文件
1893     file := excelize.NewFile()
1894     // 定义数据
1895     employees := []Employee{
1896         {1, "Alice", "Smith", 28, "Manager"},
1897         {2, "Bob", "Johnson", 35, "Engineer"},
1898         {3, "Charlie", "Wilson", 22, "Intern"},
1899     }
1900     // 创建第一个工作表
1901     sheet1 := "Employees"
1902     index1 := file.NewSheet(sheet1)
1903     // 设置工作表的列标题
1904     titles := map[string]string{
1905         "A1": "ID",
1906         "B1": "operation_id",
1907         "C1": "drone_id",
1908         "D1": "field_id",
1909         "E1": "operation_time",
1910     }
1911     for cell, title := range titles {
1912         file.SetCellValue(sheet1, cell, title)
1913     }
1914     // 写入数据到工作表
1915     for i, emp := range employees {
1916         rowIndex := i + 2 // 数据从第二行开始
1917         file.SetCellValue(sheet1,
1918             "A"+excelize.ToAlphaString(1)+excelize.ToSliceIndex(rowIndex), emp.ID)
1919         file.SetCellValue(sheet1,
1920             "B"+excelize.ToAlphaString(2)+excelize.ToSliceIndex(rowIndex), emp.operation_id)
1921         file.SetCellValue(sheet1,
1922             "C"+excelize.ToAlphaString(3)+excelize.ToSliceIndex(rowIndex), emp.drone_id)
1923         file.SetCellValue(sheet1,
1924             "D"+excelize.ToAlphaString(4)+excelize.ToSliceIndex(rowIndex), emp.field_id)
1925         file.SetCellValue(sheet1,
1926             "E"+excelize.ToAlphaString(5)+excelize.ToSliceIndex(rowIndex), emp.operation_time)
1927         file.SetCellValue(sheet1,
1928             "E"+excelize.ToAlphaString(6)+excelize.ToSliceIndex(rowIndex), emp.admin)
1929     }
1930     // 自动调整列宽
1931     for col := 'A'; col <= 'E'; col++ {
1932         colStr := string(col)
1933         file.SetColWidth(sheet1, colStr, colStr, 15)
1934     }
1935     // 设置第一个工作表为活动工作表
1936     file.SetActiveSheet(index1)
1937     // 创建第二个工作表
1938     sheet2 := "MoreData"

```

```

1939     index2 := file.NewSheet(sheet2)
1940
1941     // 保存文件
1942     filePath := "output.xlsx"
1943     if err := file.SaveAs(filePath); err != nil {
1944         log.Fatalf("Save Excel file error: %v", err)
1945     }
1946     // 提示文件已保存
1947     log.Printf("Excel file '%s' has been saved successfully.\n", filePath)
1948 }
1949 // FDF 导出
1950 func (c spray_operationController) ExportFDF() revel.Result {
1951     var spray_operation []spray_operation
1952     c.DB.Find(&spray_operation)
1953     // TODO: 实现 FDF 导出逻辑
1954     var spray_operation []Menu1
1955     if err := c.DB.Find(&spray_operation).Error; err != nil {
1956         revel.AppLog.Errorf("查询数据库失败: %v", err)
1957         c.Flash.Error("查询数据库失败, 请稍后重试")
1958         return c.Redirect(Menu1Controller.List)
1959     }
1960     // 创建 PDF 文档对象
1961     pdf := gofpdf.New("P", "mm", "A4", "")
1962     pdf.AddPage()
1963     // 设置字体
1964     pdf.SetFont("Arial", "", 12)
1965     // 添加表头
1966     pdf.CellFormat(10, 7, "序号", 1, 0, "C", false, 0, "")
1967     pdf.CellFormat(30, 7, "operation_id", 1, 0, "C", false, 0, "")//作业编号
1968     pdf.CellFormat(30, 7, "drone_id", 1, 0, "C", false, 0, "")//无人机编号
1969     pdf.CellFormat(30, 7, "field_id", 1, 0, "C", false, 0, "")//地块编号
1970     pdf.CellFormat(30, 7, "operation_time", 1, 1, "C", false, 0, "")//作业时间
1971     // 添加数据行
1972     for _, menu := range spray_operation {
1973         pdf.CellFormat(10, 7, strconv.Itoa(int(menu.序号)), 1, 0, "C", false, 0, "")
1974         pdf.CellFormat(30, 7, menu.operation_id, 1, 0, "", false, 0, "")//作业编号
1975         pdf.CellFormat(30, 7, menu.drone_id, 1, 0, "", false, 0, "")//无人机编号
1976         pdf.CellFormat(30, 7, menu.field_id, 1, 0, "", false, 0, "")//地块编号
1977         pdf.CellFormat(30, 7, menu.operation_time, 1, 1, "", false, 0, "")//作业时
1978     }
1979     // 保存 PDF 文件
1980     filePath := "path/to/save/yourfile.pdf"
1981     err := pdf.OutputFileAndClose(filePath)
1982     if err != nil {
1983         revel.AppLog.Errorf("保存 PDF 文件失败: %v", err)
1984         c.Flash.Error("保存 PDF 文件失败, 请稍后重试")
1985         return c.Redirect(Menu1Controller.List)
1986     }
1987     // 返回 PDF 文件路径供下载
1988     c.Response.ContentType = "application/pdf"
1989     c.Response.Out.Header().Set("Content-Disposition", "attachment;

```

```

1990 filename="+filePath)
1991     return c.File(filePath)
1992     return c.Render(spray_operation)
1993 }
1994 package models
1995 import "github.com/jinzhu/gorm"
1996 // 定义菜单1数据模型
1997 type spray_operation struct {
1998     gorm.Model
1999     序号      int      `gorm:"column:序号;not null"`
2000     operation_id string `gorm:"column:operation_id;not null"`//作业编号
2001     drone_id   string `gorm:"column:drone_id;not null"`//无人机编号
2002     field_id   string `gorm:"column:field_id"`//地块编号
2003     operation_time string `gorm:"column:字段拼音 4"`//作业时间
2004     操作者     string `gorm:"column:操作者"`
2005 }
2006 func init() {
2007     // 路由映射
2008     revel.Router.
2009         Route("/spray_operation/list/{page}",
2010 &controllers.spray_operationController{}).
2011         Route("/spray_operation/add", &controllers.spray_operationController{}).
2012         Route("/spray_operation/edit", &controllers.spray_operationController{}).
2013         Route("/spray_operation/delete/{id}",
2014 &controllers.spray_operationController{}).
2015         Route("/spray_operation/search/{keyword}/{page}",
2016 &controllers.spray_operationController{}).
2017         Route("/spray_operation/import", &controllers.spray_operationController{}).
2018         Route("/spray_operation/export", &controllers.spray_operationController{}).
2019 }
2020 //气象环境数据监测模块的 go lang 代码
2021 package controllers
2022 import (
2023     "log"
2024     "os"
2025     "github.com/revel/revel"
2026     "github.com/360EntSecGroup-Skylar/excelize/v2"
2027     "github.com/jinzhu/gorm"
2028     "github.com/jinzhu/gorm/dialects/mysql"
2029     "github.com/revel/revel"
2030     "github.com/jinzhu/gorm"
2031     "github.com/jung-kurt/gofpdf"
2032     "your-app-name/app/controllers"
2033     "github.com/revel/revel"
2034     _ "github.com/jinzhu/gorm/dialects/postgres"
2035     "github.com/jinzhu/gorm"
2036 )
2037 func InitDB() {
2038     var err error
2039     // Read database configuration from Revel configuration
2040     dbConfig := revel.Config.StringDefault("db.url",

```



```

2041 "postgres://localhost:5432/mydatabase")
2042 // Open a connection to the database
2043 DB, err = gorm.Open("postgres", dbConfig)
2044 if err != nil {
2045     revel.AppLog.Fatalf("Failed to connect to database: %v", err)
2046 }
2047 // Enable logging if needed
2048 DB.LogMode(true)
2049 }
2050 func CloseDB() {
2051     if DB != nil {
2052         if err := DB.Close(); err != nil {
2053             revel.AppLog.Errorf("Failed to close database connection: %v", err)
2054         }
2055     }
2056 }
2057 // 气象环境数据监测管理控制器
2058 type weather_dataController struct {
2059     *revel.Controller
2060     DB *gorm.DB // 数据库连接对象
2061 }
2062 // 初始化数据库连接
2063 func (c weather_dataController) Begin() revel.Result {
2064     db, err := gorm.Open("mysql",
2065         "user:password@/dbname?charset=utf8&parseTime=True&loc=Local")
2066     if err != nil {
2067         revel.AppLog.Error("Failed to connect database")
2068         panic(err)
2069     }
2070     c.DB = db
2071     return nil
2072 }
2073 // 结束时关闭数据库连接
2074 func (c weather_dataController) End() revel.Result {
2075     err := c.DB.Close()
2076     if err != nil {
2077         revel.AppLog.Error("Failed to close database connection")
2078     }
2079     return nil
2080 }
2081 // 列表展示
2082 func (c weather_dataController) List(page int) revel.Result {
2083     var weather_data []weather_data
2084     c.DB.Offset((page - 1) * PageSize).Limit(PageSize).Find(&weather_data)
2085     return c.Render(weather_data)
2086 }
2087 // 数据添加
2088 func (c weather_dataController) Add(menu weather_data) revel.Result {
2089     c.Validation.Required(menu.Field1).Message("气象编号不能为空")
2090     c.Validation.Required(menu.Field2).Message("地块编号不能为空")
2091     if c.Validation.HasErrors() {

```

```

2092         c.Validation.Keep()
2093         c.FlashParams()
2094         return c.Redirect(weather_dataController.List)
2095     }
2096     c.DB.Create(&menu)
2097     return c.Redirect(weather_dataController.List)
2098 }
2099 // 数据编辑
2100 func (c weather_dataController) Edit(menu weather_data) revel.Result {
2101     c.DB.Save(&menu)
2102     return c.Redirect(weather_dataController.List)
2103 }
2104 // 数据删除
2105 func (c weather_dataController) Delete(id int) revel.Result {
2106     var menu weather_data
2107     c.DB.First(&menu, id)
2108     c.DB.Delete(&menu)
2109     return c.Redirect(weather_dataController.List)
2110 }
2111 // 数据分页
2112 const PageSize = 10
2113 // 数据查询
2114 func (c weather_dataController) Search(keyword string, page int) revel.Result {
2115     var weather_data []weather_data
2116     c.DB.Where(" 字 段 1 LIKE ?", "%"+keyword+"%").Offset((page - 1) *
2117     PageSize).Limit(PageSize).Find(&weather_data)
2118     return c.Render(weather_data)
2119 }
2120 // Excel 导入
2121 func (c weather_dataController) ImportExcel(file revel.File) revel.Result {
2122     f, err := excelize.OpenReader(file)
2123     if err != nil {
2124         revel.AppLog.Error("Failed to open Excel file")
2125         return c.Redirect(weather_dataController.List)
2126     }
2127     // 读取并处理 Excel 数据
2128     // TODO: 实现 Excel 数据处理逻辑
2129     if file == nil {
2130         c.Flash.Error("请选择要导入的 Excel 文件")
2131         return c.Redirect(MenuController.List)
2132     }
2133     // 打开 Excel 文件
2134     f, err := excelize.OpenReader(file)
2135     if err != nil {
2136         revel.AppLog.Errorf("打开 Excel 文件失败: %v", err)
2137         c.Flash.Error("打开 Excel 文件失败, 请检查文件格式")
2138         return c.Redirect(MenuController.List)
2139     }
2140     // 获取工作表名称
2141     sheetName := f.GetSheetName(1) // 假设只有一个工作表, 索引从 1 开始
2142     rows, err := f.GetRows(sheetName)

```

```

2143     if err != nil {
2144         revel.AppLog.Errorf("获取 Excel 工作表行数据失败: %v", err)
2145         c.Flash.Error("获取 Excel 工作表行数据失败, 请检查文件内容")
2146         return c.Redirect(MenuController.List)
2147     }
2148     var weather_data []Menu1
2149     for rowIndex, row := range rows {
2150         if rowIndex == 0 {
2151             continue // 跳过表头
2152         }
2153         if len(row) < 5 {
2154             revel.AppLog.Errorf("Excel 行数据格式不正确: %v", row)
2155             continue // 跳过格式不正确的行
2156         }
2157         // 解析 Excel 数据
2158         menu := Menu1{
2159             序号:    rowIndex, // 假设序号是自动生成的
2160             weather_id: row[0], // 气象编号
2161             field_id: row[1], // 地块编号
2162             temperature: row[2], // 温度(℃)
2163             wind_speed: row[3], // 风速(m/s)
2164             操作者: "admin", // 假设当前操作者是管理员
2165         }
2166         // 将数据添加到列表
2167         weather_data = append(weather_data, menu)
2168     }
2169     // 将数据批量插入数据库
2170     if err := c.DB.Create(&weather_data).Error; err != nil {
2171         revel.AppLog.Errorf("批量插入 Excel 数据到数据库失败: %v", err)
2172         c.Flash.Error("导入 Excel 数据到数据库失败, 请稍后重试")
2173         return c.Redirect(MenuController.List)
2174     }
2175     c.Flash.Success("成功导入 Excel 数据")
2176     return c.Redirect(MenuController.List)
2177     return c.Redirect(weather_dataController.List)
2178 }
2179 //Excel 数据导出
2180 func Excelout() {
2181     // 创建一个新的 Excel 文件
2182     file := excelize.NewFile()
2183     // 定义数据
2184     employees := []Employee{
2185         {1, "Alice", "Smith", 28, "Manager"},
2186         {2, "Bob", "Johnson", 35, "Engineer"},
2187         {3, "Charlie", "Wilson", 22, "Intern"},
2188     }
2189     // 创建第一个工作表
2190     sheet1 := "Employees"
2191     index1 := file.NewSheet(sheet1)
2192     // 设置工作表的列标题
2193     titles := map[string]string{

```

```

2194         "A1": "ID",
2195         "B1": "weather_id",
2196         "C1": "field_id",
2197         "D1": "temperature",
2198         "E1": "wind_speed",
2199     }
2200     for cell, title := range titles {
2201         file.SetCellValue(sheet1, cell, title)
2202     }
2203     // 写入数据到工作表
2204     for i, emp := range employees {
2205         rowIndex := i + 2 // 数据从第二行开始
2206         file.SetCellValue(sheet1,
2207             "A"+excelize.ToAlphaString(1)+excelize.ToSliceIndex(rowIndex), emp.ID)
2208         file.SetCellValue(sheet1,
2209             "B"+excelize.ToAlphaString(2)+excelize.ToSliceIndex(rowIndex), emp.weather_id)
2210         file.SetCellValue(sheet1,
2211             "C"+excelize.ToAlphaString(3)+excelize.ToSliceIndex(rowIndex), emp.field_id)
2212         file.SetCellValue(sheet1,
2213             "D"+excelize.ToAlphaString(4)+excelize.ToSliceIndex(rowIndex), emp.temperature)
2214         file.SetCellValue(sheet1,
2215             "E"+excelize.ToAlphaString(5)+excelize.ToSliceIndex(rowIndex), emp.wind_speed)
2216         file.SetCellValue(sheet1,
2217             "E"+excelize.ToAlphaString(6)+excelize.ToSliceIndex(rowIndex), emp.admin)
2218     }
2219     // 自动调整列宽
2220     for col := 'A'; col <= 'E'; col++ {
2221         colStr := string(col)
2222         file.SetColWidth(sheet1, colStr, colStr, 15)
2223     }
2224     // 设置第一个工作表为活动工作表
2225     file.SetActiveSheet(index1)
2226     // 创建第二个工作表
2227     sheet2 := "MoreData"
2228     index2 := file.NewSheet(sheet2)
2229
2230     // 保存文件
2231     filePath := "output.xlsx"
2232     if err := file.SaveAs(filePath); err != nil {
2233         log.Fatalf("Save Excel file error: %v", err)
2234     }
2235     // 提示文件已保存
2236     log.Printf("Excel file '%s' has been saved successfully.\n", filePath)
2237 }
2238 // FDF 导出
2239 func (c weather_dataController) ExportFDF() revel.Result {
2240     var weather_data []weather_data
2241     c.DB.Find(&weather_data)
2242     // TODO: 实现 FDF 导出逻辑
2243     var weather_data []Menu1
2244     if err := c.DB.Find(&weather_data).Error; err != nil {

```

```

2245         revel.AppLog.Errorf("查询数据库失败: %v", err)
2246         c.Flash.Error("查询数据库失败, 请稍后重试")
2247         return c.Redirect(MenuController.List)
2248     }
2249     // 创建 PDF 文档对象
2250     pdf := gofpdf.New("P", "mm", "A4", "")
2251     pdf.AddPage()
2252     // 设置字体
2253     pdf.SetFont("Arial", "", 12)
2254     // 添加表头
2255     pdf.CellFormat(10, 7, "序号", 1, 0, "C", false, 0, "")
2256     pdf.CellFormat(30, 7, "weather_id", 1, 0, "C", false, 0, "")//气象编号
2257     pdf.CellFormat(30, 7, "field_id", 1, 0, "C", false, 0, "")//地块编号
2258     pdf.CellFormat(30, 7, "temperature", 1, 0, "C", false, 0, "")//温度(℃)
2259     pdf.CellFormat(30, 7, "wind_speed", 1, 1, "C", false, 0, "")//风速(m/s)
2260     // 添加数据行
2261     for _, menu := range weather_data {
2262         pdf.CellFormat(10, 7, strconv.Itoa(int(menu.序号)), 1, 0, "C", false, 0, "")
2263         pdf.CellFormat(30, 7, menu.weather_id, 1, 0, "", false, 0, "")//气象编号
2264         pdf.CellFormat(30, 7, menu.field_id, 1, 0, "", false, 0, "")//地块编号
2265         pdf.CellFormat(30, 7, menu.temperature, 1, 0, "", false, 0, "")//温度(℃)
2266         pdf.CellFormat(30, 7, menu.wind_speed, 1, 1, "", false, 0, "")//风速(m/s)    }
2267     // 保存 PDF 文件
2268     filePath := "path/to/save/yourfile.pdf"
2269     err := pdf.OutputFileAndClose(filePath)
2270     if err != nil {
2271         revel.AppLog.Errorf("保存 PDF 文件失败: %v", err)
2272         c.Flash.Error("保存 PDF 文件失败, 请稍后重试")
2273         return c.Redirect(MenuController.List)
2274     }
2275     // 返回 PDF 文件路径供下载
2276     c.Response.ContentType = "application/pdf"
2277     c.Response.Out.Header().Set("Content-Disposition", "attachment;
2278 filename="+filePath)
2279     return c.File(filePath)
2280     return c.Render(weather_data)
2281 }
2282 package models
2283 import "github.com/jinzhu/gorm"
2284 // 定义菜单 1 数据模型
2285 type weather_data struct {
2286     gorm.Model
2287     序号      int      `gorm:"column:序号;not null"`
2288     weather_id string `gorm:"column:weather_id;not null"`//气象编号
2289     field_id  string `gorm:"column:field_id;not null"`//地块编号
2290     temperature string `gorm:"column:temperature"`//温度(℃)
2291     wind_speed string `gorm:"column:字段拼音 4"`//风速(m/s)
2292     操作者    string `gorm:"column:操作者"`
2293 }
2294 func init() {
2295     // 路由映射

```

```

2296     revel.Router.
2297         Route("/weather_data/list/{page}", &controllers.weather_dataController{}).
2298         Route("/weather_data/add", &controllers.weather_dataController{}).
2299         Route("/weather_data/edit", &controllers.weather_dataController{}).
2300         Route("/weather_data/delete/{id}", &controllers.weather_dataController{}).
2301         Route("/weather_data/search/{keyword}/{page}",
2302 &controllers.weather_dataController{}).
2303         Route("/weather_data/import", &controllers.weather_dataController{}).
2304         Route("/weather_data/export", &controllers.weather_dataController{}).
2305     }
2306 //一键适配参数配置模块的 go lang 代码
2307 package controllers
2308 import (
2309     "log"
2310     "os"
2311     "github.com/revel/revel"
2312     "github.com/360EntSecGroup-Skylar/excelize/v2"
2313     "github.com/jinzhu/gorm"
2314     "github.com/jinzhu/gorm/dialects/mysql"
2315     "github.com/revel/revel"
2316     "github.com/jinzhu/gorm"
2317     "github.com/jung-kurt/gofpdf"
2318     "your-app-name/app/controllers"
2319     "github.com/revel/revel"
2320     _ "github.com/jinzhu/gorm/dialects/postgres"
2321     "github.com/jinzhu/gorm"
2322 )
2323 func InitDB() {
2324     var err error
2325     // Read database configuration from Revel configuration
2326     dbConfig := revel.Config.StringDefault("db.url",
2327 "postgres://localhost:5432/mydatabase")
2328     // Open a connection to the database
2329     DB, err = gorm.Open("postgres", dbConfig)
2330     if err != nil {
2331         revel.AppLog.Fatalf("Failed to connect to database: %v", err)
2332     }
2333     // Enable logging if needed
2334     DB.LogMode(true)
2335 }
2336 func CloseDB() {
2337     if DB != nil {
2338         if err := DB.Close(); err != nil {
2339             revel.AppLog.Errorf("Failed to close database connection: %v", err)
2340         }
2341     }
2342 }
2343 // 一键适配参数配置管理控制器
2344 type parameter_configController struct {
2345     *revel.Controller
2346     DB *gorm.DB // 数据库连接对象

```

```
2347 }
2348 // 初始化数据库连接
2349 func (c parameter_configController) Begin() revel.Result {
2350     db, err := gorm.Open("mysql",
2351         "user:password@/dbname?charset=utf8&parseTime=True&loc=Local")
2352     if err != nil {
2353         revel.AppLog.Error("Failed to connect database")
2354         panic(err)
2355     }
2356     c.DB = db
2357     return nil
2358 }
2359 // 结束时关闭数据库连接
2360 func (c parameter_configController) End() revel.Result {
2361     err := c.DB.Close()
2362     if err != nil {
2363         revel.AppLog.Error("Failed to close database connection")
2364     }
2365     return nil
2366 }
2367 // 列表展示
2368 func (c parameter_configController) List(page int) revel.Result {
2369     var parameter_config []parameter_config
2370     c.DB.Offset((page - 1) * PageSize).Limit(PageSize).Find(&parameter_config)
2371     return c.Render(parameter_config)
2372 }
2373 // 数据添加
2374 func (c parameter_configController) Add(menu parameter_config) revel.Result {
2375     c.Validation.Required(menu.Field1).Message("配置编号不能为空")
2376     c.Validation.Required(menu.Field2).Message("生长阶段不能为空")
2377     if c.Validation.HasErrors() {
2378         c.Validation.Keep()
2379         c.FlashParams()
2380         return c.Redirect(parameter_configController.List)
2381     }
2382     c.DB.Create(&menu)
2383     return c.Redirect(parameter_configController.List)
2384 }
2385 // 数据编辑
2386 func (c parameter_configController) Edit(menu parameter_config) revel.Result {
2387     c.DB.Save(&menu)
2388     return c.Redirect(parameter_configController.List)
2389 }
2390 // 数据删除
2391 func (c parameter_configController) Delete(id int) revel.Result {
2392     var menu parameter_config
2393     c.DB.First(&menu, id)
2394     c.DB.Delete(&menu)
2395     return c.Redirect(parameter_configController.List)
2396 }
2397 // 数据分页
```

```

2398 const PageSize = 10
2399 // 数据查询
2400 func (c parameter_configController) Search(keyword string, page int) revel.Result {
2401     var parameter_config []parameter_config
2402     c.DB.Where("字段 1 LIKE ?", "%"+keyword+"%").Offset((page - 1) *
2403     PageSize).Limit(PageSize).Find(&parameter_config)
2404     return c.Render(parameter_config)
2405 }
2406 // Excel 导入
2407 func (c parameter_configController) ImportExcel(file revel.File) revel.Result {
2408     f, err := excelize.OpenReader(file)
2409     if err != nil {
2410         revel.AppLog.Error("Failed to open Excel file")
2411         return c.Redirect(parameter_configController.List)
2412     }
2413     // 读取并处理 Excel 数据
2414     // TODO: 实现 Excel 数据处理逻辑
2415     if file == nil {
2416         c.Flash.Error("请选择要导入的 Excel 文件")
2417         return c.Redirect(MenuController.List)
2418     }
2419     // 打开 Excel 文件
2420     f, err := excelize.OpenReader(file)
2421     if err != nil {
2422         revel.AppLog.Errorf("打开 Excel 文件失败: %v", err)
2423         c.Flash.Error("打开 Excel 文件失败, 请检查文件格式")
2424         return c.Redirect(MenuController.List)
2425     }
2426     // 获取工作表名称
2427     sheetName := f.GetSheetName(1) // 假设只有一个工作表, 索引从 1 开始
2428     rows, err := f.GetRows(sheetName)
2429     if err != nil {
2430         revel.AppLog.Errorf("获取 Excel 工作表行数据失败: %v", err)
2431         c.Flash.Error("获取 Excel 工作表行数据失败, 请检查文件内容")
2432         return c.Redirect(MenuController.List)
2433     }
2434     var parameter_config []Menu1
2435     for rowIndex, row := range rows {
2436         if rowIndex == 0 {
2437             continue // 跳过表头
2438         }
2439         if len(row) < 5 {
2440             revel.AppLog.Errorf("Excel 行数据格式不正确: %v", row)
2441             continue // 跳过格式不正确的行
2442         }
2443         // 解析 Excel 数据
2444         menu := Menu1{
2445             序号:    rowIndex, // 假设序号是自动生成的
2446             config_id: row[0], // 配置编号
2447             growth_stage: row[1], // 生长阶段
2448             pesticide_recommend: row[2], // 推荐农药

```



```

2449         flight_speed: row[3], //飞行速度(m/s)
2450         操作者: "admin", // 假设当前操作者是管理员
2451     }
2452     // 将数据添加到列表
2453     parameter_config = append(parameter_config, menu)
2454 }
2455 // 将数据批量插入数据库
2456 if err := c.DB.Create(&parameter_config).Error; err != nil {
2457     revel.AppLog.Errorf("批量插入 Excel 数据到数据库失败: %v", err)
2458     c.Flash.Error("导入 Excel 数据到数据库失败, 请稍后重试")
2459     return c.Redirect(Menu1Controller.List)
2460 }
2461 c.Flash.Success("成功导入 Excel 数据")
2462 return c.Redirect(Menu1Controller.List)
2463 return c.Redirect(parameter_configController.List)
2464 }
2465 //Excel 数据导出
2466 func Excelout() {
2467     // 创建一个新的 Excel 文件
2468     file := excelize.NewFile()
2469     // 定义数据
2470     employees := []Employee{
2471         {1, "Alice", "Smith", 28, "Manager"},
2472         {2, "Bob", "Johnson", 35, "Engineer"},
2473         {3, "Charlie", "Wilson", 22, "Intern"},
2474     }
2475     // 创建第一个工作表
2476     sheet1 := "Employees"
2477     index1 := file.NewSheet(sheet1)
2478     // 设置工作表的列标题
2479     titles := map[string]string{
2480         "A1": "ID",
2481         "B1": "config_id",
2482         "C1": "growth_stage",
2483         "D1": "pesticide_recommend",
2484         "E1": "flight_speed",
2485     }
2486     for cell, title := range titles {
2487         file.SetCellValue(sheet1, cell, title)
2488     }
2489     // 写入数据到工作表
2490     for i, emp := range employees {
2491         rowIndex := i + 2 // 数据从第二行开始
2492         file.SetCellValue(sheet1,
2493             "A"+excelize.ToAlphaString(1)+excelize.ToSliceIndex(rowIndex), emp.ID)
2494         file.SetCellValue(sheet1,
2495             "B"+excelize.ToAlphaString(2)+excelize.ToSliceIndex(rowIndex), emp.config_id)
2496         file.SetCellValue(sheet1,
2497             "C"+excelize.ToAlphaString(3)+excelize.ToSliceIndex(rowIndex), emp.growth_stage)
2498         file.SetCellValue(sheet1,
2499             "D"+excelize.ToAlphaString(4)+excelize.ToSliceIndex(rowIndex),

```

```

2500 emp.pesticide_recommend)
2501     file.SetCellValue(sheet1,
2502 "E"+excelize.ToAlphaString(5)+excelize.ToSliceIndex(rowIndex), emp.flight_speed)
2503 file.SetCellValue(sheet1,
2504 "E"+excelize.ToAlphaString(6)+excelize.ToSliceIndex(rowIndex), emp.admin)
2505     }
2506     // 自动调整列宽
2507     for col := 'A'; col <= 'E'; col++ {
2508         colStr := string(col)
2509         file.SetColWidth(sheet1, colStr, colStr, 15)
2510     }
2511     // 设置第一个工作表为活动工作表
2512     file.SetActiveSheet(index1)
2513     // 创建第二个工作表
2514     sheet2 := "MoreData"
2515     index2 := file.NewSheet(sheet2)
2516
2517     // 保存文件
2518     filePath := "output.xlsx"
2519     if err := file.SaveAs(filePath); err != nil {
2520         log.Fatalf("Save Excel file error: %v", err)
2521     }
2522     // 提示文件已保存
2523     log.Printf("Excel file '%s' has been saved successfully.\n", filePath)
2524 }
2525 // FDF 导出
2526 func (c parameter_configController) ExportFDF() revel.Result {
2527     var parameter_config []parameter_config
2528     c.DB.Find(&parameter_config)
2529     // TODO: 实现 FDF 导出逻辑
2530     var parameter_config []Menu1
2531     if err := c.DB.Find(&parameter_config).Error; err != nil {
2532         revel.AppLog.Errorf("查询数据库失败: %v", err)
2533         c.Flash.Error("查询数据库失败, 请稍后重试")
2534         return c.Redirect(Menu1Controller.List)
2535     }
2536     // 创建 PDF 文档对象
2537     pdf := gofpdf.New("P", "mm", "A4", "")
2538     pdf.AddPage()
2539     // 设置字体
2540     pdf.SetFont("Arial", "", 12)
2541     // 添加表头
2542     pdf.CellFormat(10, 7, "序号", 1, 0, "C", false, 0, "")
2543     pdf.CellFormat(30, 7, "config_id", 1, 0, "C", false, 0, "")//配置编号
2544     pdf.CellFormat(30, 7, "growth_stage", 1, 0, "C", false, 0, "")//生长阶段
2545     pdf.CellFormat(30, 7, "pesticide_recommend", 1, 0, "C", false, 0, "")//推荐农药
2546     pdf.CellFormat(30, 7, "flight_speed", 1, 1, "C", false, 0, "")//飞行速度(m/s)
2547     // 添加数据行
2548     for _, menu := range parameter_config {
2549         pdf.CellFormat(10, 7, strconv.Itoa(int(menu.序号)), 1, 0, "C", false, 0, "")
2550         pdf.CellFormat(30, 7, menu.config_id, 1, 0, "", false, 0, "")//配置编号

```

```

2551         pdf.Format(30, 7, menu.growth_stage, 1, 0, "", false, 0, "")//生长阶段
2552         pdf.Format(30, 7, menu.pesticide_recommend, 1, 0, "", false, 0, "")//推
2553 荐农药
2554         pdf.Format(30, 7, menu.flight_speed, 1, 1, "", false, 0, "")//飞行速度
2555     (m/s)    }
2556     // 保存 PDF 文件
2557     filePath := "path/to/save/yourfile.pdf"
2558     err := pdf.OutputFileAndClose(filePath)
2559     if err != nil {
2560         revel.AppLog.Errorf("保存 PDF 文件失败: %v", err)
2561         c.Flash.Error("保存 PDF 文件失败, 请稍后重试")
2562         return c.Redirect(MenuController.List)
2563     }
2564     // 返回 PDF 文件路径供下载
2565     c.Response.ContentType = "application/pdf"
2566     c.Response.Out.Header().Set("Content-Disposition", "attachment;
2567 filename="+filePath)
2568     return c.File(filePath)
2569     return c.Render(parameter_config)
2570 }
2571 package models
2572 import "github.com/jinzhu/gorm"
2573 // 定义菜单 1 数据模型
2574 type parameter_config struct {
2575     gorm.Model
2576     序号      int      `gorm:"column:序号;not null"`
2577     config_id  string  `gorm:"column:config_id;not null"`//配置编号
2578     growth_stage  string  `gorm:"column:growth_stage;not null"`//生长阶段
2579     pesticide_recommend  string  `gorm:"column:pesticide_recommend"`//推荐农药
2580     flight_speed  string  `gorm:"column:字段拼音 4"`//飞行速度(m/s)
2581     操作者      string  `gorm:"column:操作者"`
2582 }
2583 func init() {
2584     // 路由映射
2585     revel.Router.
2586         Route("/parameter_config/list/{page}",
2587 &controllers.parameter_configController{}).
2588         Route("/parameter_config/add", &controllers.parameter_configController{}).
2589         Route("/parameter_config/edit", &controllers.parameter_configController{}).
2590         Route("/parameter_config/delete/{id}",
2591 &controllers.parameter_configController{}).
2592         Route("/parameter_config/search/{keyword}/{page}",
2593 &controllers.parameter_configController{}).
2594         Route("/parameter_config/import",
2595 &controllers.parameter_configController{}).
2596         Route("/parameter_config/export",
2597 &controllers.parameter_configController{}).
2598     }
2599 //操作员日志记录模块的 go 语言代码
2600 package controllers
2601 import (

```

```

2602 "log"
2603     "os"
2604     "github.com/revel/revel"
2605     "github.com/360EntSecGroup-Skylar/excelize/v2"
2606     "github.com/jinzhu/gorm"
2607     "github.com/jinzhu/gorm/dialects/mysql"
2608     "github.com/revel/revel"
2609     "github.com/jinzhu/gorm"
2610     "github.com/jung-kurt/gofpdf"
2611     "your-app-name/app/controllers"
2612 "github.com/revel/revel"
2613 _ "github.com/jinzhu/gorm/dialects/postgres"
2614 "github.com/jinzhu/gorm"
2615 )
2616 func InitDB() {
2617     var err error
2618     // Read database configuration from Revel configuration
2619     dbConfig := revel.Config.StringDefault("db.url",
2620 "postgres://localhost:5432/mydatabase")
2621     // Open a connection to the database
2622     DB, err = gorm.Open("postgres", dbConfig)
2623     if err != nil {
2624         revel.AppLog.Fatalf("Failed to connect to database: %v", err)
2625     }
2626     // Enable logging if needed
2627     DB.LogMode(true)
2628 }
2629 func CloseDB() {
2630     if DB != nil {
2631         if err := DB.Close(); err != nil {
2632             revel.AppLog.Errorf("Failed to close database connection: %v", err)
2633         }
2634     }
2635 }
2636 // 操作员日志记录管理控制器
2637 type operator_logController struct {
2638     *revel.Controller
2639     DB *gorm.DB // 数据库连接对象
2640 }
2641 // 初始化数据库连接
2642 func (c operator_logController) Begin() revel.Result {
2643     db, err := gorm.Open("mysql",
2644 "user:password@/dbname?charset=utf8&parseTime=True&loc=Local")
2645     if err != nil {
2646         revel.AppLog.Error("Failed to connect database")
2647         panic(err)
2648     }
2649     c.DB = db
2650     return nil
2651 }
2652 // 结束时关闭数据库连接

```

```

2653 func (c operator_logController) End() revel.Result {
2654     err := c.DB.Close()
2655     if err != nil {
2656         revel.AppLog.Error("Failed to close database connection")
2657     }
2658     return nil
2659 }
2660 // 列表展示
2661 func (c operator_logController) List(page int) revel.Result {
2662     var operator_log []operator_log
2663     c.DB.Offset((page - 1) * PageSize).Limit(PageSize).Find(&operator_log)
2664     return c.Render(operator_log)
2665 }
2666 // 数据添加
2667 func (c operator_logController) Add(menu operator_log) revel.Result {
2668     c.Validation.Required(menu.Field1).Message("日志编号不能为空")
2669     c.Validation.Required(menu.Field2).Message("操作员姓名不能为空")
2670     if c.Validation.HasErrors() {
2671         c.Validation.Keep()
2672         c.FlashParams()
2673         return c.Redirect(operator_logController.List)
2674     }
2675     c.DB.Create(&menu)
2676     return c.Redirect(operator_logController.List)
2677 }
2678 // 数据编辑
2679 func (c operator_logController) Edit(menu operator_log) revel.Result {
2680     c.DB.Save(&menu)
2681     return c.Redirect(operator_logController.List)
2682 }
2683 // 数据删除
2684 func (c operator_logController) Delete(id int) revel.Result {
2685     var menu operator_log
2686     c.DB.First(&menu, id)
2687     c.DB.Delete(&menu)
2688     return c.Redirect(operator_logController.List)
2689 }
2690 // 数据分页
2691 const PageSize = 10
2692 // 数据查询
2693 func (c operator_logController) Search(keyword string, page int) revel.Result {
2694     var operator_log []operator_log
2695     c.DB.Where(" 字 段 1 LIKE ?", "%"+keyword+"%").Offset((page - 1) *
2696     PageSize).Limit(PageSize).Find(&operator_log)
2697     return c.Render(operator_log)
2698 }
2699 // Excel 导入
2700 func (c operator_logController) ImportExcel(file revel.File) revel.Result {
2701     f, err := excelize.OpenReader(file)
2702     if err != nil {
2703         revel.AppLog.Error("Failed to open Excel file")

```

```

2704         return c.Redirect(operator_logController.List)
2705     }
2706     // 读取并处理 Excel 数据
2707     // TODO: 实现 Excel 数据处理逻辑
2708     if file == nil {
2709         c.Flash.Error("请选择要导入的 Excel 文件")
2710         return c.Redirect(Menu1Controller.List)
2711     }
2712     // 打开 Excel 文件
2713     f, err := excelize.OpenReader(file)
2714     if err != nil {
2715         revel.AppLog.Errorf("打开 Excel 文件失败: %v", err)
2716         c.Flash.Error("打开 Excel 文件失败, 请检查文件格式")
2717         return c.Redirect(Menu1Controller.List)
2718     }
2719     // 获取工作表名称
2720     sheetName := f.GetSheetName(1) // 假设只有一个工作表, 索引从 1 开始
2721     rows, err := f.GetRows(sheetName)
2722     if err != nil {
2723         revel.AppLog.Errorf("获取 Excel 工作表行数据失败: %v", err)
2724         c.Flash.Error("获取 Excel 工作表行数据失败, 请检查文件内容")
2725         return c.Redirect(Menu1Controller.List)
2726     }
2727     var operator_log []Menu1
2728     for rowIndex, row := range rows {
2729         if rowIndex == 0 {
2730             continue // 跳过表头
2731         }
2732         if len(row) < 5 {
2733             revel.AppLog.Errorf("Excel 行数据格式不正确: %v", row)
2734             continue // 跳过格式不正确的行
2735         }
2736         // 解析 Excel 数据
2737         menu := Menu1{
2738             序号:    rowIndex, // 假设序号是自动生成的
2739             log_id:  row[0], // 日志编号
2740             operator_name: row[1], // 操作员姓名
2741             action_type: row[2], // 操作类型
2742             timestamp: row[3], // 操作时间
2743             操作者: "admin", // 假设当前操作者是管理员
2744         }
2745         // 将数据添加到列表
2746         operator_log = append(operator_log, menu)
2747     }
2748     // 将数据批量插入数据库
2749     if err := c.DB.Create(&operator_log).Error; err != nil {
2750         revel.AppLog.Errorf("批量插入 Excel 数据到数据库失败: %v", err)
2751         c.Flash.Error("导入 Excel 数据到数据库失败, 请稍后重试")
2752         return c.Redirect(Menu1Controller.List)
2753     }
2754     c.Flash.Success("成功导入 Excel 数据")

```

```

2755     return c.Redirect(Menu1Controller.List)
2756     return c.Redirect(operator_logController.List)
2757 }
2758 //Excel 数据导出
2759 func Excelout() {
2760     // 创建一个新的 Excel 文件
2761     file := excelize.NewFile()
2762     // 定义数据
2763     employees := []Employee{
2764         {1, "Alice", "Smith", 28, "Manager"},
2765         {2, "Bob", "Johnson", 35, "Engineer"},
2766         {3, "Charlie", "Wilson", 22, "Intern"},
2767     }
2768     // 创建第一个工作表
2769     sheet1 := "Employees"
2770     index1 := file.NewSheet(sheet1)
2771     // 设置工作表的列标题
2772     titles := map[string]string{
2773         "A1": "ID",
2774         "B1": "log_id",
2775         "C1": "operator_name",
2776         "D1": "action_type",
2777         "E1": "timestamp",
2778     }
2779     for cell, title := range titles {
2780         file.SetCellValue(sheet1, cell, title)
2781     }
2782     // 写入数据到工作表
2783     for i, emp := range employees {
2784         rowIndex := i + 2 // 数据从第二行开始
2785         file.SetCellValue(sheet1,
2786             "A"+excelize.ToAlphaString(1)+excelize.ToSliceIndex(rowIndex), emp.ID)
2787         file.SetCellValue(sheet1,
2788             "B"+excelize.ToAlphaString(2)+excelize.ToSliceIndex(rowIndex), emp.log_id)
2789         file.SetCellValue(sheet1,
2790             "C"+excelize.ToAlphaString(3)+excelize.ToSliceIndex(rowIndex), emp.operator_name)
2791         file.SetCellValue(sheet1,
2792             "D"+excelize.ToAlphaString(4)+excelize.ToSliceIndex(rowIndex), emp.action_type)
2793         file.SetCellValue(sheet1,
2794             "E"+excelize.ToAlphaString(5)+excelize.ToSliceIndex(rowIndex), emp.timestamp)
2795         file.SetCellValue(sheet1,
2796             "E"+excelize.ToAlphaString(6)+excelize.ToSliceIndex(rowIndex), emp.admin)
2797     }
2798     // 自动调整列宽
2799     for col := 'A'; col <= 'E'; col++ {
2800         colStr := string(col)
2801         file.SetColWidth(sheet1, colStr, colStr, 15)
2802     }
2803     // 设置第一个工作表为活动工作表
2804     file.SetActiveSheet(index1)
2805     // 创建第二个工作表

```

```

2806     sheet2 := "MoreData"
2807     index2 := file.NewSheet(sheet2)
2808
2809     // 保存文件
2810     filePath := "output.xlsx"
2811     if err := file.SaveAs(filePath); err != nil {
2812         log.Fatalf("Save Excel file error: %v", err)
2813     }
2814     // 提示文件已保存
2815     log.Printf("Excel file '%s' has been saved successfully.\n", filePath)
2816 }
2817 // FDF 导出
2818 func (c operator_logController) ExportFDF() revel.Result {
2819     var operator_log []operator_log
2820     c.DB.Find(&operator_log)
2821     // TODO: 实现 FDF 导出逻辑
2822     var operator_log []Menu1
2823     if err := c.DB.Find(&operator_log).Error; err != nil {
2824         revel.AppLog.Errorf("查询数据库失败: %v", err)
2825         c.Flash.Error("查询数据库失败, 请稍后重试")
2826         return c.Redirect(Menu1Controller.List)
2827     }
2828     // 创建 PDF 文档对象
2829     pdf := gofpdf.New("P", "mm", "A4", "")
2830     pdf.AddPage()
2831     // 设置字体
2832     pdf.SetFont("Arial", "", 12)
2833     // 添加表头
2834     pdf.CellFormat(10, 7, "序号", 1, 0, "C", false, 0, "")
2835     pdf.CellFormat(30, 7, "log_id", 1, 0, "C", false, 0, "")//日志编号
2836     pdf.CellFormat(30, 7, "operator_name", 1, 0, "C", false, 0, "")//操作员姓名
2837     pdf.CellFormat(30, 7, "action_type", 1, 0, "C", false, 0, "")//操作类型
2838     pdf.CellFormat(30, 7, "timestamp", 1, 1, "C", false, 0, "")//操作时间
2839     // 添加数据行
2840     for _, menu := range operator_log {
2841         pdf.CellFormat(10, 7, strconv.Itoa(int(menu.序号)), 1, 0, "C", false, 0, "")
2842         pdf.CellFormat(30, 7, menu.log_id, 1, 0, "", false, 0, "")//日志编号
2843         pdf.CellFormat(30, 7, menu.operator_name, 1, 0, "", false, 0, "")//操作员姓
2844         名
2845         pdf.CellFormat(30, 7, menu.action_type, 1, 0, "", false, 0, "")//操作类型
2846         pdf.CellFormat(30, 7, menu.timestamp, 1, 1, "", false, 0, "")//操作时间    }
2847     // 保存 PDF 文件
2848     filePath := "path/to/save/yourfile.pdf"
2849     err := pdf.OutputFileAndClose(filePath)
2850     if err != nil {
2851         revel.AppLog.Errorf("保存 PDF 文件失败: %v", err)
2852         c.Flash.Error("保存 PDF 文件失败, 请稍后重试")
2853         return c.Redirect(Menu1Controller.List)
2854     }
2855     // 返回 PDF 文件路径供下载
2856     c.Response.ContentType = "application/pdf"

```



```

2857     c.Response.Out.Header().Set("Content-Disposition",           "attachment;
2858     filename="+filePath)
2859     return c.File(filePath)
2860     return c.Render(operator_log)
2861 }
2862 package models
2863 import "github.com/jinzhu/gorm"
2864 // 定义菜单1数据模型
2865 type operator_log struct {
2866     gorm.Model
2867     序号      int      `gorm:"column:序号;not null"`
2868     log_id    string `gorm:"column:log_id;not null"`//日志编号
2869     operator_name string `gorm:"column:operator_name;not null"`//操作员姓名
2870     action_type string `gorm:"column:action_type"`//操作类型
2871     timestamp string `gorm:"column:字段拼音4"`//操作时间
2872     操作者    string `gorm:"column:操作者"`
2873 }
2874 func init() {
2875     // 路由映射
2876     revel.Router.
2877         Route("/operator_log/list/{page}", &controllers.operator_logController{}).
2878         Route("/operator_log/add", &controllers.operator_logController{}).
2879         Route("/operator_log/edit", &controllers.operator_logController{}).
2880         Route("/operator_log/delete/{id}", &controllers.operator_logController{}).
2881         Route("/operator_log/search/{keyword}/{page}",
2882         &controllers.operator_logController{}).
2883         Route("/operator_log/import", &controllers.operator_logController{}).
2884         Route("/operator_log/export", &controllers.operator_logController{}).
2885 }
2886 //农情分析报告管理模块的 go语言 代码
2887 package controllers
2888 import (
2889     "log"
2890     "os"
2891     "github.com/revel/revel"
2892     "github.com/360EntSecGroup-Skylar/excelize/v2"
2893     "github.com/jinzhu/gorm"
2894     "github.com/jinzhu/gorm/dialects/mysql"
2895     "github.com/revel/revel"
2896     "github.com/jinzhu/gorm"
2897     "github.com/jung-kurt/gofpdf"
2898     "your-app-name/app/controllers"
2899     "github.com/revel/revel"
2900     _ "github.com/jinzhu/gorm/dialects/postgres"
2901     "github.com/jinzhu/gorm"
2902 )
2903 func InitDB() {
2904     var err error
2905     // Read database configuration from Revel configuration
2906     dbConfig := revel.Config.StringDefault("db.url",
2907     "postgres://localhost:5432/mydatabase")

```

```

2908 // Open a connection to the database
2909 DB, err = gorm.Open("postgres", dbConfig)
2910 if err != nil {
2911     revel.AppLog.Fatalf("Failed to connect to database: %v", err)
2912 }
2913 // Enable logging if needed
2914 DB.LogMode(true)
2915 }
2916 func CloseDB() {
2917     if DB != nil {
2918         if err := DB.Close(); err != nil {
2919             revel.AppLog.Errorf("Failed to close database connection: %v", err)
2920         }
2921     }
2922 }
2923 // 农情分析报告管理管理控制器
2924 type analysis_reportController struct {
2925     *revel.Controller
2926     DB *gorm.DB // 数据库连接对象
2927 }
2928 // 初始化数据库连接
2929 func (c analysis_reportController) Begin() revel.Result {
2930     db, err := gorm.Open("mysql",
2931 "user:password@/dbname?charset=utf8&parseTime=True&loc=Local")
2932     if err != nil {
2933         revel.AppLog.Error("Failed to connect database")
2934         panic(err)
2935     }
2936     c.DB = db
2937     return nil
2938 }
2939 // 结束时关闭数据库连接
2940 func (c analysis_reportController) End() revel.Result {
2941     err := c.DB.Close()
2942     if err != nil {
2943         revel.AppLog.Error("Failed to close database connection")
2944     }
2945     return nil
2946 }
2947 // 列表展示
2948 func (c analysis_reportController) List(page int) revel.Result {
2949     var analysis_report []analysis_report
2950     c.DB.Offset((page - 1) * PageSize).Limit(PageSize).Find(&analysis_report)
2951     return c.Render(analysis_report)
2952 }
2953 // 数据添加
2954 func (c analysis_reportController) Add(menu analysis_report) revel.Result {
2955     c.Validation.Required(menu.Field1).Message("报告编号不能为空")
2956     c.Validation.Required(menu.Field2).Message("地块编号不能为空")
2957     if c.Validation.HasErrors() {
2958         c.Validation.Keep()

```

```

2959         c.FlashParams()
2960         return c.Redirect(analysis_reportController.List)
2961     }
2962     c.DB.Create(&menu)
2963     return c.Redirect(analysis_reportController.List)
2964 }
2965 // 数据编辑
2966 func (c analysis_reportController) Edit(menu analysis_report) revel.Result {
2967     c.DB.Save(&menu)
2968     return c.Redirect(analysis_reportController.List)
2969 }
2970 // 数据删除
2971 func (c analysis_reportController) Delete(id int) revel.Result {
2972     var menu analysis_report
2973     c.DB.First(&menu, id)
2974     c.DB.Delete(&menu)
2975     return c.Redirect(analysis_reportController.List)
2976 }
2977 // 数据分页
2978 const PageSize = 10
2979 // 数据查询
2980 func (c analysis_reportController) Search(keyword string, page int) revel.Result {
2981     var analysis_report []analysis_report
2982     c.DB.Where(" 字 段 1 LIKE ?", "%"+keyword+"%").Offset((page - 1) *
2983     PageSize).Limit(PageSize).Find(&analysis_report)
2984     return c.Render(analysis_report)
2985 }
2986 // Excel 导入
2987 func (c analysis_reportController) ImportExcel(file revel.File) revel.Result {
2988     f, err := excelize.OpenReader(file)
2989     if err != nil {
2990         revel.AppLog.Error("Failed to open Excel file")
2991         return c.Redirect(analysis_reportController.List)
2992     }
2993     // 读取并处理 Excel 数据
2994     // TODO: 实现 Excel 数据处理逻辑
2995     if file == nil {
2996         c.Flash.Error("请选择要导入的 Excel 文件")
2997         return c.Redirect(MenuController.List)
2998     }
2999     // 打开 Excel 文件
3000     f, err := excelize.OpenReader(file)
3001     if err != nil {
3002         revel.AppLog.Errorf("打开 Excel 文件失败: %v", err)
3003         c.Flash.Error("打开 Excel 文件失败, 请检查文件格式")
3004         return c.Redirect(MenuController.List)
3005     }
3006     // 获取工作表名称
3007     sheetName := f.GetSheetName(1) // 假设只有一个工作表, 索引从 1 开始
3008     rows, err := f.GetRows(sheetName)
3009     if err != nil {

```

```

3010     revel.AppLog.Errorf("获取 Excel 工作表行数据失败: %v", err)
3011     c.Flash.Error("获取 Excel 工作表行数据失败, 请检查文件内容")
3012     return c.Redirect(Menu1Controller.List)
3013 }
3014 var analysis_report []Menu1
3015 for rowIndex, row := range rows {
3016     if rowIndex == 0 {
3017         continue // 跳过表头
3018     }
3019     if len(row) < 5 {
3020         revel.AppLog.Errorf("Excel 行数据格式不正确: %v", row)
3021         continue // 跳过格式不正确的行
3022     }
3023     // 解析 Excel 数据
3024     menu := Menu1{
3025         序号:    rowIndex, // 假设序号是自动生成的
3026         report_id: row[0], // 报告编号
3027         field_id: row[1], // 地块编号
3028         report_title: row[2], // 报告标题
3029         generate_date: row[3], // 生成日期
3030         操作者: "admin", // 假设当前操作者是管理员
3031     }
3032     // 将数据添加到列表
3033     analysis_report = append(analysis_report, menu)
3034 }
3035 // 将数据批量插入数据库
3036 if err := c.DB.Create(&analysis_report).Error; err != nil {
3037     revel.AppLog.Errorf("批量插入 Excel 数据到数据库失败: %v", err)
3038     c.Flash.Error("导入 Excel 数据到数据库失败, 请稍后重试")
3039     return c.Redirect(Menu1Controller.List)
3040 }
3041 c.Flash.Success("成功导入 Excel 数据")
3042 return c.Redirect(Menu1Controller.List)
3043 return c.Redirect(analysis_reportController.List)
3044 }
3045 //Excel 数据导出
3046 func Excelout() {
3047     // 创建一个新的 Excel 文件
3048     file := excelize.NewFile()
3049     // 定义数据
3050     employees := []Employee{
3051         {1, "Alice", "Smith", 28, "Manager"},
3052         {2, "Bob", "Johnson", 35, "Engineer"},
3053         {3, "Charlie", "Wilson", 22, "Intern"},
3054     }
3055     // 创建第一个工作表
3056     sheet1 := "Employees"
3057     index1 := file.NewSheet(sheet1)
3058     // 设置工作表的列标题
3059     titles := map[string]string{
3060         "A1": "ID",

```

```

3061         "B1": "report_id",
3062         "C1": "field_id",
3063         "D1": "report_title",
3064         "E1": "generate_date",
3065     }
3066     for cell, title := range titles {
3067         file.SetCellValue(sheet1, cell, title)
3068     }
3069     // 写入数据到工作表
3070     for i, emp := range employees {
3071         rowIndex := i + 2 // 数据从第二行开始
3072         file.SetCellValue(sheet1,
3073             "A"+excelize.ToAlphaString(1)+excelize.ToSliceIndex(rowIndex), emp.ID)
3074         file.SetCellValue(sheet1,
3075             "B"+excelize.ToAlphaString(2)+excelize.ToSliceIndex(rowIndex), emp.report_id)
3076         file.SetCellValue(sheet1,
3077             "C"+excelize.ToAlphaString(3)+excelize.ToSliceIndex(rowIndex), emp.field_id)
3078         file.SetCellValue(sheet1,
3079             "D"+excelize.ToAlphaString(4)+excelize.ToSliceIndex(rowIndex), emp.report_title)
3080         file.SetCellValue(sheet1,
3081             "E"+excelize.ToAlphaString(5)+excelize.ToSliceIndex(rowIndex), emp.generate_date)
3082         file.SetCellValue(sheet1,
3083             "E"+excelize.ToAlphaString(6)+excelize.ToSliceIndex(rowIndex), emp.admin)
3084     }
3085     // 自动调整列宽
3086     for col := 'A'; col <= 'E'; col++ {
3087         colStr := string(col)
3088         file.SetColWidth(sheet1, colStr, colStr, 15)
3089     }
3090     // 设置第一个工作表为活动工作表
3091     file.SetActiveSheet(index1)
3092     // 创建第二个工作表
3093     sheet2 := "MoreData"
3094     index2 := file.NewSheet(sheet2)
3095
3096     // 保存文件
3097     filePath := "output.xlsx"
3098     if err := file.SaveAs(filePath); err != nil {
3099         log.Fatalf("Save Excel file error: %v", err)
3100     }
3101     // 提示文件已保存
3102     log.Printf("Excel file '%s' has been saved successfully.\n", filePath)
3103 }
3104 // FDF 导出
3105 func (c analysis_reportController) ExportFDF() revel.Result {
3106     var analysis_report []analysis_report
3107     c.DB.Find(&analysis_report)
3108     // TODO: 实现 FDF 导出逻辑
3109     var analysis_report []Menu1
3110     if err := c.DB.Find(&analysis_report).Error; err != nil {
3111         revel.AppLog.Errorf("查询数据库失败: %v", err)

```

```

3112         c.Flash.Error("查询数据库失败, 请稍后重试")
3113         return c.Redirect(MenuController.List)
3114     }
3115     // 创建 PDF 文档对象
3116     pdf := gofpdf.New("P", "mm", "A4", "")
3117     pdf.AddPage()
3118     // 设置字体
3119     pdf.SetFont("Arial", "", 12)
3120     // 添加表头
3121     pdf.CellFormat(10, 7, "序号", 1, 0, "C", false, 0, "")
3122     pdf.CellFormat(30, 7, "report_id", 1, 0, "C", false, 0, "")//报告编号
3123     pdf.CellFormat(30, 7, "field_id", 1, 0, "C", false, 0, "")//地块编号
3124     pdf.CellFormat(30, 7, "report_title", 1, 0, "C", false, 0, "")//报告标题
3125     pdf.CellFormat(30, 7, "generate_date", 1, 1, "C", false, 0, "")//生成日期
3126     // 添加数据行
3127     for _, menu := range analysis_report {
3128         pdf.CellFormat(10, 7, strconv.Itoa(int(menu.序号)), 1, 0, "C", false, 0, "")
3129         pdf.CellFormat(30, 7, menu.report_id, 1, 0, "", false, 0, "")//报告编号
3130         pdf.CellFormat(30, 7, menu.field_id, 1, 0, "", false, 0, "")//地块编号
3131         pdf.CellFormat(30, 7, menu.report_title, 1, 0, "", false, 0, "")//报告标题
3132         pdf.CellFormat(30, 7, menu.generate_date, 1, 1, "", false, 0, "")//生成日期
3133     }
3134     // 保存 PDF 文件
3135     filePath := "path/to/save/yourfile.pdf"
3136     err := pdf.OutputFileAndClose(filePath)
3137     if err != nil {
3138         revel.AppLog.Errorf("保存 PDF 文件失败: %v", err)
3139         c.Flash.Error("保存 PDF 文件失败, 请稍后重试")
3140         return c.Redirect(MenuController.List)
3141     }
3142     // 返回 PDF 文件路径供下载
3143     c.Response.ContentType = "application/pdf"
3144     c.Response.Out.Header().Set("Content-Disposition", "attachment;
3145 filename="+filePath)
3146     return c.File(filePath)
3147     return c.Render(analysis_report)
3148 }
3149 package models
3150 import "github.com/jinzhu/gorm"
3151 // 定义菜单 1 数据模型
3152 type analysis_report struct {
3153     gorm.Model
3154     序号      int      `gorm:"column:序号;not null"`
3155     report_id string `gorm:"column:report_id;not null"`//报告编号
3156     field_id  string `gorm:"column:field_id;not null"`//地块编号
3157     report_title string `gorm:"column:report_title"`//报告标题
3158     generate_date string `gorm:"column:字段拼音 4"`//生成日期
3159     操作者    string `gorm:"column:操作者"`
3160 }
3161 func init() {
3162     // 路由映射

```

```

3163     revel.Router.
3164         Route("/analysis_report/list/{page}",
3165 &controllers.analysis_reportController{}).
3166         Route("/analysis_report/add", &controllers.analysis_reportController{}).
3167         Route("/analysis_report/edit", &controllers.analysis_reportController{}).
3168         Route("/analysis_report/delete/{id}",
3169 &controllers.analysis_reportController{}).
3170         Route("/analysis_report/search/{keyword}/{page}",
3171 &controllers.analysis_reportController{}).
3172         Route("/analysis_report/import", &controllers.analysis_reportController{}).
3173         Route("/analysis_report/export", &controllers.analysis_reportController{}).
3174     }
3175 <!doctype html>
3176 <html lang="en">
3177 <head>
3178 <meta charset="utf-8">
3179 <meta name="viewport" content="width=device-width, initial-scale=1,
3180 shrink-to-fit=no">
3181 <meta name="description" content="">
3182 <meta name="author" content="">
3183 <link rel="icon" href="favicon.ico">
3184 <title></title>
3185 <link rel="stylesheet" href="css/simplebar.css">
3186 <link rel="stylesheet" href="css/feather.css">
3187 <link rel="stylesheet" href="css/select2.css">
3188 <link rel="stylesheet" href="css/jquery.steps.css">
3189 <link rel="stylesheet" href="css/jquery.timepicker.css">
3190 <link rel="stylesheet" href="css/quill.snow.css">
3191 <link rel="stylesheet" href="css/daterangepicker.css">
3192 <link rel="stylesheet" href="css/app-light.css" id="lightTheme" disabled>
3193 <link rel="stylesheet" href="css/app-dark.css" id="darkTheme">
3194 </head>
3195 <body class="vertical dark ">
3196 <div class="wrapper">
3197 <nav class="topnav navbar navbar-light">
3198 <button type="button" class="navbar-toggler text-muted mt-2 p-0 mr-3 collapseSidebar">
3199 <i class="fe fe-menu navbar-toggler-icon"></i>
3200 </button>
3201 <form class="form-inline mr-auto searchform text-muted">
3202 <input class="form-control mr-sm-2 bg-transparent border-0 pl-4 text-muted"
3203 type="search"
3204 placeholder="Type something..." id="sss" aria-label="Search">
3205 </form>
3206 <ul class="nav">
3207 <li class="nav-item">
3208 <a class="nav-link text-muted my-2" href="/#" id="modeSwitcher"
3209 data-mode="dark">
3210 <i class="fe fe-sun fe-l6"></i>
3211 </a>
3212 </li>
3213 <li class="nav-item">

```

```

3214     <a class="nav-link text-muted my-2" href="/#" data-toggle="modal"
3215         data-target=".modal-shortcut">
3216         <span class="fe fe-grid fe-16"></span>
3217     </a>
3218 </li>
3219 <li class="nav-item nav-notif">
3220     <a class="nav-link text-muted my-2" href="/#" data-toggle="modal"
3221 data-target=".modal-notif">
3222         <span class="fe fe-bell fe-16"></span>
3223         <span class="dot dot-md bg-success"></span>
3224     </a>
3225 </li>
3226 <li class="nav-item dropdown">
3227 </li>
3228 </ul>
3229 </nav>
3230 <aside class="sidebar-left border-right bg-white shadow" id="leftSidebar"
3231 data-simplebar>
3232 <a href="#" class="btn collapseSidebar toggle-btn d-lg-none text-muted ml-2 mt-3"
3233 data-toggle="toggle">
3234 <i class="fe fe-x"><span class="sr-only"></span></i>
3235 </a>
3236 <nav class="vertnav navbar navbar-light">
3237 <div class="w-100 mb-4 d-flex">
3238 <a class="navbar-brand mx-auto mt-2 flex-fill text-center" href="/index.html">
3239 </a>
3240 </div>
3241 <ul class="navbar-nav flex-fill w-100 mb-2">
3242 <li class="nav-item dropdown">
3243 <a href="#ui-elements" data-toggle="collapse" aria-expanded="true"
3244 class="dropdown-toggle nav-link">
3245 <i class="fe fe-box fe-16"></i>
3246 <span class="ml-3 item-text">管理菜单</span>
3247 </a>
3248 <ul class="collapse list-unstyled pl-4 w-100" style="display: block;"
3249 id="ui-elements">
3250 </ul>
3251 </li>
3252 </li>
3253 </ul>
3254 <div class="btn-box w-100 mt-4 mb-1">
3255 <button type="button" onclick="location.href='login.html'" class="btn mb-2
3256 btn-primary btn-lg btn-block" id="modeSwitcher" data-mode="dark">
3257 <i class="fe fe-shopping-cart fe-12 mr-2"></i><span class="small">退出</span>
3258 </button>
3259 </div>
3260 </nav>
3261 </aside>
3262 <main role="main" class="main-content">
3263 <div class="container-fluid">
3264 <div class="row justify-content-center">

```



```

3265     <div class="col-12">
3266         <h2 class="h3 my-3 page-title">Orders</h2>
3267         <div class="row mb-4 items-align-center">
3268             <div class="col-md">
3269                 <ul class="nav nav-pills justify-content-start">
3270                     <li class="nav-item">
3271                         <a class="nav-link active bg-transparent pr-2 pl-0
3272 text-primary" id="aaa" data-toggle="modal" data-target="#add"
3273                         >添加</a>
3274                     </li>
3275                     <li class="nav-item">
3276                         <a class="nav-link text-muted px-2" href="#">上传</a>
3277                     </li>
3278                     <li class="nav-item">
3279                         <a class="nav-link text-muted px-2" href="#">下载</a>
3280                     </li>
3281                     <li class="nav-item">
3282                         <a class="nav-link text-muted px-2" href="#">打印</a>
3283                     </li>
3284                     <li class="nav-item">
3285                         <a class="nav-link text-muted px-2" @click="clear()">清除
3286 </a>
3287                     </li>
3288                 </ul>
3289             </div>
3290         </div>
3291         <div id="add" class="modal fade modal-slide" tabindex="-1" role="dialog"
3292             aria-labelledby="defaultModalLabel" aria-hidden="true">
3293             <div class="modal-dialog" role="document" style="width: 500px;">
3294                 <div class="modal-content">
3295                     <div class="modal-header">
3296                         <h5 class="modal-title" id="defaultModalLabel">添加</h5>
3297                         <button type="button" class="close" data-dismiss="modal"
3298 aria-label="Close">
3299                             <i class="fe fe-x fe-12"></i>
3300                         </button>
3301                     </div>
3302                     <div class="modal-body">
3303                         <div class="form-group">
3304                             <label for="inputEmail">数据 1</label>
3305                             <input type="text" id="add1" class="form-control
3306 form-control-lg" v-model="arrayrow.data1" placeholder="" autofocus="">
3307                         </div>
3308                     </div>
3309                     <div class="modal-body">
3310                         <div class="form-group">
3311                             <label for="inputEmail">数据 2</label>
3312                             <input type="text" id="add2" class="form-control
3313 form-control-lg" v-model="arrayrow.data2" placeholder="" autofocus="">
3314                         </div>
3315                     </div>

```

```

3316         <div class="modal-body">
3317             <div class="form-group">
3318                 <label for="inputEmail" >数据 3</label>
3319                 <input type="text" id="add3" class="form-control
3320 form-control-lg" v-model="arrayrow.data3" placeholder="" autofocus="">
3321             </div>
3322         </div>
3323         <div class="modal-body">
3324             <div class="form-group">
3325                 <label for="inputEmail" >数据 4</label>
3326                 <input type="text" id="add4" class="form-control
3327 form-control-lg" v-model="arrayrow.data4" placeholder="" autofocus="">
3328             </div>
3329         </div>
3330         <div class="modal-footer">
3331             <button type="button" data-dismiss="modal" class="btn mb-2
3332 btn-secondary">取消</button>
3333             <button type="button" data-dismiss="modal" class="btn mb-2
3334 btn-primary" @click="adds()" id="aaaa">添加</button>
3335         </div>
3336     </div>
3337 </div>
3338 </div>
3339 <table class="table border table-hover">
3340     <thead>
3341         <tr class="bg-info text-white">
3342             <th><input type='checkbox' name='checkItem' /></th>
3343             <th>序号</th>
3344             <th v-for="(item, index) in listname">{{item.name}}</th>
3345             <th>添加日期</th>
3346             <th>最后一次操作人</th>
3347             <th>操作</th>
3348         </tr>
3349     </thead>
3350     <tbody>
3351         <tr>
3352             <tr v-for="(item, index) in arrayrow" id="fff">
3353                 <td><input type="checkbox" name="checkbox2" id="checkbox2"
3354 /></td>
3355                 <td>{{index+1}}</td>
3356                 <td id="ffff">{{item.data1}}</td>
3357                 <td>{{item.data2}}</td>
3358                 <td>{{item.data3}}</td>
3359                 <td>{{item.data4}}</td>
3360                 <td class="date"></td>
3361                 <td>用户{{index+1}}</td>
3362                 <td>
3363                     <a class="tablelink" >检查</a>
3364                     <a class="tablelink" id="eee" data-toggle="modal"
3365 @click="edits(index)" href="#edit">编辑</a>
3366                     <a class="tablelink" data-bs-target="#del" id="ddd"

```

```

3367 data-toggle="modal" href="#del">删除</a>
3368         </td>
3369     </tr>
3370 </tbody>
3371 </table>
3372 <nav aria-label="Table Paging" class="my-3">
3373     <ul class="pagination justify-content-end mb-0">
3374         <li class="page-item active"><a class="page-link" href="#">上一页
3375     </a></li>
3376         <li class="page-item"><a class="page-link" href="#">1</a></li>
3377         <li class="page-item"><a class="page-link" href="#">2</a></li>
3378         <li class="page-item"><a class="page-link" href="#">3</a></li>
3379         <li class="page-item"><a class="page-link" href="#">下一页</a></li>
3380     </ul>
3381 </nav>
3382 </div>
3383 </div>
3384 </div>
3385 <div class="modal fade" id="del" tabindex="-1" role="dialog"
3386     aria-labelledby="exampleModalCenterTitle" aria-hidden="true">
3387     <div class="modal-dialog modal-dialog-centered" style="margin: 150px auto;"
3388     role="document">
3389         <div class="modal-content">
3390             <div class="modal-header">
3391                 <h5 class="modal-title" id="exampleModalCenterTitle">删除</h5>
3392                 <button type="button" class="close" data-dismiss="modal"
3393                     aria-label="Close">
3394                     <span aria-hidden="true">&times;</span>
3395                 </button>
3396             </div>
3397             <div class="modal-body">
3398                 确定删除此信息?
3399             </div>
3400             <div class="modal-footer">
3401                 <button type="button" class="btn btn-secondary"
3402                     data-dismiss="modal">取消</button>
3403                 <button type="button" class="btn btn-primary"
3404                     data-dismiss="modal" @click="delates(index)" id="dddd">确定
3405             </button>
3406             </div>
3407         </div>
3408     </div>
3409 </div>
3410 <div id="edit" class="modal fade modal-slide" tabindex="-1" role="dialog"
3411     aria-labelledby="defaultModalLabel" aria-hidden="true">
3412     <div class="modal-dialog" role="document" style="width: 500px;">
3413         <div class="modal-content">
3414             <div class="modal-header">
3415                 <h5 class="modal-title" id="defaultModalLabel">修改</h5>
3416                 <button type="button" class="close" data-dismiss="modal"
3417                 aria-label="Close">

```

```

3418         <i class="fe fe-x fe-12"></i>
3419     </button>
3420 </div>
3421 <div class="modal-body">
3422     <div class="form-group">
3423         <label for="inputEmail" >数据 1</label>
3424         <input type="text" id="edit1" class="form-control form-control-lg"
3425 v-model="arrlist.data1" placeholder="" autofocus="">
3426     </div>
3427 </div>
3428 <div class="modal-body">
3429     <div class="form-group">
3430         <label for="inputEmail" >数据 2</label>
3431         <input type="text" id="edit2" class="form-control form-control-lg"
3432 v-model="arrlist.data2" placeholder="" autofocus="">
3433     </div>
3434 </div>
3435 <div class="modal-body">
3436     <div class="form-group">
3437         <label for="inputEmail">数据 3</label>
3438         <input type="text" id="edit3" class="form-control form-control-lg"
3439 v-model="arrlist.data3" placeholder="" autofocus="">
3440     </div>
3441 </div>
3442 <div class="modal-body">
3443     <div class="form-group">
3444         <label for="inputEmail" >数据 4</label>
3445         <input type="text" id="edit4" class="form-control form-control-lg"
3446 v-model="arrlist.data4" placeholder="" autofocus="">
3447     </div>
3448 </div>
3449 <input type="hidden" id="id" class="form-control form-control-lg"
3450 placeholder="" autofocus="">
3451 <div class="modal-footer">
3452     <button type="button" data-dismiss="modal" class="btn mb-2
3453 btn-secondary">取消</button>
3454     <button type="button" data-dismiss="modal" class="btn mb-2 btn-primary"
3455 @click="saves()" id="eeee">修改</button>
3456 </div>
3457 </div>
3458 </div>
3459 </div>
3460 <div class="modal fade modal-shortcut modal-slide" tabindex="-1" role="dialog"
3461 aria-labelledby="defaultModalLabel" aria-hidden="true">
3462 <div class="modal-dialog" role="document">
3463     <div class="modal-content">
3464         <div class="modal-header">
3465             <h5 class="modal-title" id="defaultModalLabel">Shortcuts</h5>
3466             <button type="button" class="close" data-dismiss="modal"
3467 aria-label="Close">
3468                 <span aria-hidden="true">&times;</span>

```

```

3469         </button>
3470     </div>
3471     <div class="modal-body px-5">
3472         <div class="row align-items-center">
3473             <div class="col-6 text-center">
3474                 <div class="squirrel bg-success justify-content-center">
3475                     <i class="fe fe-cpu fe-32 align-self-center text-white"></i>
3476                 </div>
3477                 <p>控制区</p>
3478             </div>
3479             <div class="col-6 text-center">
3480                 <div class="squirrel bg-primary justify-content-center">
3481                     <i class="fe fe-activity fe-32 align-self-center
3482 text-white"></i>
3483                 </div>
3484                 <p>活动</p>
3485             </div>
3486         </div>
3487         <div class="row align-items-center">
3488             <div class="col-6 text-center">
3489                 <div class="squirrel bg-primary justify-content-center">
3490                     <i class="fe fe-droplet fe-32 align-self-center
3491 text-white"></i>
3492                 </div>
3493                 <p>下载</p>
3494             </div>
3495             <div class="col-6 text-center">
3496                 <div class="squirrel bg-primary justify-content-center">
3497                     <i class="fe fe-upload-cloud fe-32 align-self-center
3498 text-white"></i>
3499                 </div>
3500                 <p>上传</p>
3501             </div>
3502         </div>
3503         <div class="row align-items-center">
3504             <div class="col-6 text-center">
3505                 <div class="squirrel bg-primary justify-content-center">
3506                     <i class="fe fe-users fe-32 align-self-center
3507 text-white"></i>
3508                 </div>
3509                 <p>用户</p>
3510             </div>
3511             <div class="col-6 text-center">
3512                 <div class="squirrel bg-primary justify-content-center">
3513                     <i class="fe fe-settings fe-32 align-self-center
3514 text-white"></i>
3515                 </div>
3516                 <p>设置</p>
3517             </div>
3518         </div>
3519     </div>

```

```
3520     </div>
3521 </div>
3522 </div>
3523 </main>
3524 </div>
3525 <script src="js/jquery.min.js" type="text/javascript"></script>
3526 <script src="js/popper.min.js" type="text/javascript"></script>
3527 <script src="js/moment.min.js" type="text/javascript"></script>
3528 <script src="js/bootstrap.min.js" type="text/javascript"></script>
3529 <script src="js/simplebar.min.js" type="text/javascript"></script>
3530 <script src='js/daterangepicker.js' type="text/javascript"></script>
3531 <script src="js/tinycolor-min.js" type="text/javascript"></script>
3532 <script src="js/config.js" type="text/javascript"></script>
3533 <script src='js/jquery.mask.min.js' type="text/javascript"></script>
3534 <script src='js/select2.min.js' type="text/javascript"></script>
3535 <script src='js/jquery.steps.min.js' type="text/javascript"></script>
3536 <script src='js/jquery.validate.min.js' type="text/javascript"></script>
3537 <script src='js/jquery.timepicker.js' type="text/javascript"></script>
3538 <script src='js/quill.min.js' type="text/javascript"></script>
3539 <script type="text/javascript">
3540 $.select2'.select2({
3541   theme: 'bootstrap4',
3542 });
3543 $.select2-multi'.select2({
3544   multiple: true,
3545   theme: 'bootstrap4',
3546 });
3547 $.drgpicker').daterangepicker({
3548   singleDatePicker: true,
3549   timePicker: false,
3550   showDropdowns: true,
3551   locale: {
3552     format: 'MM/DD/YYYY'
3553   }
3554 });
3555 $.time-input').timepicker({
3556   'scrollDefault': 'now'
3557 });
3558 /** date range picker */
3559 if ($.datetimes').length) {
3560   $.datetimes').daterangepicker({
3561     timePicker: true,
3562     startDate: moment().startOf('hour'),
3563     endDate: moment().startOf('hour').add(32, 'hour'),
3564     locale: {
3565       format: 'M/DD hh:mm A'
3566     }
3567   });
3568 }
3569 var start = moment().subtract(29, 'days');
3570 var end = moment();
```

```

3571 function cb(start, end) {
3572   $(' #reportrange span').html(start.format(' MMMM D, YYYY') + ' - ' + end.format(' MMMM D,
3573   YYYY'));
3574   }
3575   $(' #reportrange').daterangepicker({
3576     startDate: start,
3577     endDate: end,
3578     ranges: {
3579       'Today': [moment(), moment()],
3580       'Yesterday': [moment().subtract(1, 'days'), moment().subtract(1, 'days')],
3581       'Last 7 Days': [moment().subtract(6, 'days'), moment()],
3582       'Last 30 Days': [moment().subtract(29, 'days'), moment()],
3583       'This Month': [moment().startOf('month'), moment().endOf('month')],
3584       'Last Month': [moment().subtract(1, 'month').startOf('month'), moment().subtract(1,
3585       'month').endOf(
3586         'month')]
3587     }
3588   }, cb);
3589   cb(start, end);
3590   $(' .input-placeholder').mask("00/00/0000", {
3591     placeholder: "__/__/____"
3592   });
3593   $(' .input-zip').mask('00000-000', {
3594     placeholder: "____-____"
3595   });
3596   $(' .input-money').mask("#.##0,00", {
3597     reverse: true
3598   });
3599   $(' .input-phoneus').mask('(000) 000-0000');
3600   $(' .input-mixed').mask('AAA 000-S0S');
3601   $(' .input-ip').mask('0ZZ.0ZZ.0ZZ.0ZZ', {
3602     translation: {
3603       'Z': {
3604         pattern: /[0-9]/,
3605         optional: true
3606       }
3607     },
3608     placeholder: "____.____.____.____"
3609   });
3610   // editor
3611   var editor = document.getElementById('editor');
3612   if (editor) {
3613     var toolbarOptions = [
3614       [{
3615         'font': []
3616       }],
3617       [{
3618         'header': [1, 2, 3, 4, 5, 6, false]
3619       }],
3620       ['bold', 'italic', 'underline', 'strike'],
3621       ['blockquote', 'code-block'],

```

```

3622  [{
3623      'header': 1
3624  },
3625  {
3626      'header': 2
3627  }
3628  ],
3629  [{
3630      'list': 'ordered'
3631  },
3632  {
3633      'list': 'bullet'
3634  }
3635  ],
3636  [{
3637      'script': 'sub'
3638  },
3639  {
3640      'script': 'super'
3641  }
3642  ],
3643  [{
3644      'indent': '-1'
3645  },
3646  {
3647      'indent': '+1'
3648  }
3649  ], // outdent/indent
3650  [{
3651      'direction': 'rtl'
3652  }], // text direction
3653  [{
3654      'color': []
3655  },
3656  {
3657      'background': []
3658  }
3659  ], // dropdown with defaults from theme
3660  [{
3661      'align': []
3662  }],
3663  ['clean'] // remove formatting button
3664  ];
3665  var quill = new Quill(editor, {
3666      modules: {
3667          toolbar: toolbarOptions
3668      },
3669      theme: 'snow'
3670  });
3671  }
3672  // Example starter JavaScript for disabling form submissions if there are invalid fields

```



```

3673 (function() {
3674 'use strict';
3675 window.addEventListener('load', function() {
3676 // Fetch all the forms we want to apply custom Bootstrap validation styles to
3677 var forms = document.getElementsByClassName('needs-validation');
3678 // Loop over them and prevent submission
3679 var validation = Array.prototype.filter.call(forms, function(form) {
3680     form.addEventListener('submit', function(event) {
3681         if (form.checkValidity() === false) {
3682             event.preventDefault();
3683             event.stopPropagation();
3684         }
3685         form.classList.add('was-validated');
3686     }, false);
3687 });
3688 }, false);
3689 })();
3690 </script>
3691 <script src="js/apps.js" type="text/javascript"></script>
3692 <script src="config/config.js"></script>
3693 <script src="config/data.js"></script>
3694 <script src="config/vue2.7.14.js"></script>
3695 <script>
3696 var listname = data2[0]
3697 var username = data[0][0].text
3698 var listdata = JSON.parse(localStorage.getItem(username));
3699 if (listdata == null) {
3700     localStorage.setItem(username, JSON.stringify(data[1]))
3701     var listdata = data[1]
3702 }
3703 $(' #ui-elements').append(ajaxnav(data[0], 0))
3704 $(' .page-title').html(username)
3705 var mydate = new Date();
3706 var date = mydate.getFullYear() + '-' + (mydate.getMonth() + 1) + '-' + mydate.getDate();
3707 $(' .date').html(date)
3708 window.vm = new Vue({
3709     el: '.main-content',
3710     data: {
3711         arrayrow: [],
3712         pageinfo: 1,
3713         listname: listname,
3714         page: 0,
3715         shdel: false,
3716         shordiel: false,
3717         shadd: false,
3718         arrlist: {
3719             data1: '',
3720             data2: '',
3721             data3: '',
3722             data4: '',
3723         }

```

```
3724 },
3725 mounted() {
3726     const that = this
3727     that.intser()
3728 },
3729 methods: {
3730     intser() {
3731         const that = this
3732         if (that.pageinfo == 1) {
3733             if (that.pageinfo == 1 && localStorage.getItem(username)) {
3734                 that.infodata = JSON.parse(localStorage.getItem(username))
3735                 that.arrayrow = that.infodata
3736                 that.page = that.infodata.length
3737             } else {
3738                 let pagell = aa.dataset
3739                 var str_jsonData = JSON.stringify(pagell);
3740                 localStorage.setItem(username, str_jsonData);
3741                 that.infodata = JSON.parse(localStorage.getItem(username))
3742                 that.arrayrow = that.infodata;
3743                 that.page = that.infodata.length
3744             }
3745         } else {
3746             // 线上的缓存数据
3747             that.infodata = JSON.parse(localStorage.getItem(username))
3748             that.arrayrow = that.infodata
3749             that.page = that.infodata.length
3750         }
3751     },
3752     add() {
3753         console.log(1111)
3754         this.shadd = true
3755     },
3756     adds() {
3757         let aa = {
3758             data1: this.arrayrow.data1,
3759             data2: this.arrayrow.data2,
3760             data3: this.arrayrow.data3,
3761             data4: this.arrayrow.data4,
3762             status: 1,
3763             date: '2023-05-24',
3764             name: '超级管理员',
3765         }
3766         this.arrayrow.unshift(aa)
3767         var str_jsonData = JSON.stringify(this.arrayrow);
3768         localStorage.setItem(username, str_jsonData);
3769         this.intser()
3770         this.arrayrow.data1 = '',
3771         this.arrayrow.data2 = '',
3772         this.arrayrow.data3 = '',
3773         this.arrayrow.data4 = ''
3774         this.shadd = false
```

```

3775     },
3776     edits(index) {
3777         this.shordiel = true
3778         this.arrrlist.data1 = this.arrayrow[index].data1
3779         this.arrrlist.data2 = this.arrayrow[index].data2
3780         this.arrrlist.data3 = this.arrayrow[index].data3
3781         this.arrrlist.data4 = this.arrayrow[index].data4
3782         this.arrrlist.index = index
3783         this.arrrlist.deleta = index
3784         console.log(this.arrrlist.index)
3785     },
3786     saves() {
3787         const that = this
3788         const fn1 = that.arrrlist.index
3789         that.arrayrow[fn1].data1 = that.arrrlist.data1
3790         that.arrayrow[fn1].data2 = that.arrrlist.data2
3791         that.arrayrow[fn1].data3 = that.arrrlist.data3
3792         that.arrayrow[fn1].data4 = that.arrrlist.data4
3793         var str_jsonData = JSON.stringify(that.arrayrow);
3794         localStorage.setItem(username, str_jsonData);
3795         this.shordiel = false
3796     },
3797     dels(index) {
3798         console.log(1111)
3799         this.shdel = true
3800         this.idd = index
3801     },
3802     delates() {
3803         console.log(222)
3804         const that = this
3805         this.shdel = false
3806         console.log(that.idd)
3807         that.arrayrow.splice(that.idd, 1)
3808         var str_jsonData = JSON.stringify(that.arrayrow);
3809         this.shdel = false
3810         localStorage.setItem(username, str_jsonData);
3811     },
3812     colse() {
3813         this.shordiel = false
3814         this.shdel = false
3815         this.shadd = false
3816     },
3817     clear() {
3818         var that = this
3819         localStorage.removeItem(username)
3820         that.infodata = JSON.parse(localStorage.getItem(username))
3821         window.location.reload()
3822     },
3823     dayin() {
3824         var cfgData = new Object;
3825         cfgData.XAction = "Printers";

```

```

3826         cfgData.XOpenId = m_OpenId;
3827         cfgData.XTokens = m_Tokens;
3828         CCPrintingAPI.Excute(cfgData,
3829             XTimeout => {},
3830             XFail => {
3831                 this.mytipsmessage = 'API 可能未安装或未启动, ' + XFail;
3832             },
3833             XSuccess => {
3834                 var d = new Date;
3835                 this.timetick = d.getTime();
3836                 var apiResult = JSON.parse(XSuccess.trim());
3837                 if (apiResult.Status == "1") {
3838                     if (apiResult.Message.length > 1) {
3839                         this.options = [];
3840                         for (var i = 1; i < apiResult.Message.length; i++) {
3841                             var op = new Object;
3842                             op.text = apiResult.Message[i];
3843                             op.value = apiResult.Message[i];
3844                             this.options.push(op);
3845                         }
3846                         /*设置上次使用的打印机*/
3847                         this.selected = this.options[0].value;
3848                     }
3849                 } else {
3850                     if (apiResult.Message.length > 1)
3851                         this.mytipsmessage = apiResult.Message[0] + " : " +
3852 apiResult.Message[1];
3853                     else
3854                         this.mytipsmessage = apiResult.Message[0];
3855                 }
3856             }
3857         );
3858     }
3859 },
3860 })
3861 function TaskManager() {
3862     let tasks = [];
3863     function addTask(description) {
3864         if (!description.trim()) return false;
3865         const newTask = {
3866             id: Date.now(),
3867             description,
3868             completed: false
3869         };
3870         tasks.push(newTask);
3871         return true;
3872     }
3873     function removeTask(taskId) {
3874         const index = tasks.findIndex(task => task.id === taskId);
3875         if (index !== -1) {
3876             tasks.splice(index, 1);

```

```
3877     return true;
3878   }
3879   return false;
3880 }
3881 function markTaskComplete(taskId) {
3882   const task = tasks.find(task => task.id === taskId);
3883   if (task) {
3884     task.completed = true;
3885     return true;
3886   }
3887   return false;
3888 }
3889 function getTasks() {
3890   return [...tasks];
3891 }
3892 function filterTasksByStatus(status) {
3893   if (status !== 'all' && status !== 'completed' && status !== 'pending') return [];
3894   return tasks.filter(task => {
3895     if (status === 'all') return true;
3896     return status === 'completed' ? task.completed : !task.completed;
3897   });
3898 }
3899 function updateTaskDescription(taskId, newDescription) {
3900   if (!newDescription.trim()) return false;
3901   const task = tasks.find(task => task.id === taskId);
3902   if (task) {
3903     task.description = newDescription;
3904     return true;
3905   }
3906   return false;
3907 }
3908 function clearCompletedTasks() {
3909   tasks = tasks.filter(task => !task.completed);
3910 }
3911 function getAllTasksCount() {
3912   return tasks.length;
3913 }
3914 function getCompletedTasksCount() {
3915   return tasks.filter(task => task.completed).length;
3916 }
3917 function getPendingTasksCount() {
3918   return tasks.filter(task => !task.completed).length;
3919 }
3920 function findTaskById(taskId) {
3921   return tasks.find(task => task.id === taskId);
3922 }
3923 function sortTasksByDate(order = 'asc') {
3924   if (order !== 'asc' && order !== 'desc') return false;
3925   tasks.sort((a, b) => {
3926     if (order === 'asc') return a.id - b.id;
3927     return b.id - a.id;
```

```
3928     });
3929     return true;
3930 }
3931 function sortTasksByCompletion(order = 'asc') {
3932     if (order !== 'asc' && order !== 'desc') return false;
3933     tasks.sort((a, b) => {
3934         if (order === 'asc') return a.completed - b.completed;
3935         return b.completed - a.completed;
3936     });
3937     return true;
3938 }
3939 function saveTasksToLocalStorage(key) {
3940     localStorage.setItem(key, JSON.stringify(tasks));
3941 }
3942 function loadTasksFromLocalStorage(key) {
3943     const storedTasks = localStorage.getItem(key);
3944     if (storedTasks) {
3945         tasks = JSON.parse(storedTasks);
3946     }
3947 }
3948 function exportTasksToJson() {
3949     return JSON.stringify(tasks, null, 2);
3950 }
3951 function importTasksFromJson(jsonString) {
3952     try {
3953         const parsedTasks = JSON.parse(jsonString);
3954         if (Array.isArray(parsedTasks)) {
3955             tasks = parsedTasks;
3956             return true;
3957         }
3958     } catch (e) {}
3959     return false;
3960 }
3961 function printTasks() {
3962     console.log('All Tasks:');
3963     tasks.forEach(task => {
3964         console.log(`ID: ${task.id}, Description: ${task.description}, Completed:
3965 ${task.completed}`);
3966     });
3967 }
3968 return {
3969     addTask,
3970     removeTask,
3971     markTaskComplete,
3972     getTasks,
3973     filterTasksByStatus,
3974     updateTaskDescription,
3975     clearCompletedTasks,
3976     getAllTasksCount,
3977     getCompletedTasksCount,
3978     getPendingTasksCount,
```

```
3979     findTaskById,
3980     sortTasksByDate,
3981     sortTasksByCompletion,
3982     saveTasksToLocalStorage,
3983     loadTasksFromLocalStorage,
3984     exportTasksToJson,
3985     importTasksFromJson,
3986     printTasks
3987   };
3988 }
3989 // Example usage:
3990 const manager = new TaskManager();
3991 manager.addTask("Buy groceries");
3992 manager.addTask("Walk the dog");
3993 manager.markTaskComplete(manager.getTasks()[0].id);
3994 manager.printTasks();
3995 console.log("Pending Tasks Count:", manager.getPendingTasksCount());
3996 console.log("Completed Tasks Count:", manager.getCompletedTasksCount());
3997 manager.saveTasksToLocalStorage("myTasks");
3998 manager.loadTasksFromLocalStorage("myTasks");
3999 manager.printTasks();
4000 </script>
4001 </body>
4002 </html>
```